IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 5 May 2024, accepted 12 May 2024, date of publication 15 May 2024, date of current version 22 May 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3401247

RESEARCH ARTICLE

A Big Data-Driven Hybrid Model for Enhancing Streaming Service Customer Retention Through Churn Prediction Integrated With Explainable AI

USMAN GANI JOY[1], KAZI EKRAMUL HOQUE[1], MOHAMMED NAZIM UDDIN[1], (Member, IEEE), LINKON CHOWDHURY[1], AND SEUNG-BO PARK[2]

[1]School of Science, Engineering & Technology, East Delta University, Chattagram 4209, Bangladesh
[2]Department of Software Convergence Engineering, Inha University, Michugol-gu, Incheon 22212, Republic of Korea

Corresponding author: Kazi Ekramul Hoque (ekramul.h@eastdelta.edu.bd)

This work was supported by the National Research Foundation of Korea (NRF) funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2023-00252952.

ABSTRACT Customer churn prediction is a critical issue that streaming services face as retaining existing subscribers is vital to the success of the business. Creating reliable churn prediction models is important because the costs of acquiring new customers are usually higher than those involved in retaining existing ones. In this study, we propose a big data-driven hybrid model combining a deep neural network with a machine-learning model to efficiently forecast customer churn. Our proposed model uses Long Short-Term Memory (LSTM) with a Gated Recurrent Unit (GRU) to capture the trends in subscribers' usage patterns over time. In addition, light gradient boosting (Light GBM) is used to leverage insights from sequential modeling along with original attributes to forecast churn. Moreover, feature selection techniques like Chi-squared testing and Sequential Feature Selection (SFS) are utilized to choose the optimum set of features for our proposed model. Furthermore, several individual models, including deep learning and traditional machine learning algorithms are also evaluated and compared with our proposed hybrid model. Additionally, the study illustrates model interpretations using Shapley Additive Explanations (SHAP) and Explainable Boosting Machine (EBM) which are used for identifying influential features in streaming services enhancing customer retention efforts. These techniques provide transparency into our proposed model's forecasting, making them more actionable and understandable for decision-makers. Extensive experimental evaluation demonstrates the hybrid model achieves best-in-class performance with 95.60% AUC and 90.09% F1 score.

INDEX TERMS Artificial intelligence, classification algorithms, deep learning, decision support systems, explainable AI, model interpretation, semi-supervised learning, big data analysis.

I. INTRODUCTION

Customer churn [1] pertains to the unfortunate situation where customers discontinue their business relationship with a company. This can be caused by a customer switching to a competitor, canceling a subscription or stopping using a product or service. Customer churn is a major concern for businesses as it can lead to decreased revenue, reduced customer lifetime value and a negative impact on brand reputation [2]. Attracting new customers can be a costly endeavor and the loss of customers translates to a depletion of valuable revenue streams [3]. Companies with high customer churn rates also face increased marketing and acquisition costs as they work to replace lost customers [3]. Harvard Business

The associate editor coordinating the review of this manuscript and approving it for publication was Kathiravan Srinivasan.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Review states that "the cost of creating a new customer is 5 to 25 times the cost of keeping an old customer" [4]. Before any business adopts a new approach, it is important to keep in mind that their customer base should not be reduced. Churn reduction is more critical than ever. According to analysts, the corporation has suffered a significant income loss as a result of consumer drop. As a result, there is a great weight and importance placed on the process of analyzing those consumers' behavior and taking the appropriate steps to solve it [1]. Minimizing customer churn necessitates proficient data analysis, tailored marketing efforts and enhanced customer service [5]. Some ways that companies can improve customer satisfaction, reduce churn rates and drive business growth are by using machine learning and other methods to predict customer behavior. By identifying customers likely to churn and revealing the reasons behind the churn, companies can take proactive measures to retain them, such as offering targeted promotions, reasonable pricing or improving customer service. Many datasets can be found for particular fields such as telecommunication to predict customer churn or customer behavior. But when it comes to the analysis of customer behavior for streaming websites or e-commerce fields, there are hardly any big datasets available on the internet. A customer log dataset [6] has been collected from the Udacity program and the course itself was created in collaboration with Appen, IBM Watson, Bertelsmann, Insight, Kaggle and Starbucks. This particular dataset is perfect for this study as it closely resembles a real-life dataset or log file of any streaming service such as Spotify, Deezer, Soundcloud, Gaana etc. The objective of this study is to create a machine-learning pipeline that can forecast customer churn in the streaming service market. This initiative is intended to assist companies in boosting their profits.

The novelty of our approach is the combination of modern deep learning techniques with traditional machine learning techniques whereas the existing literature has usually employed either deep neural network models or classic machine learning models. We use both deep learning models and conventional models such as Catboost, XGBoost and logistic regression. Moreover, we propose a unique hybrid machine-learning model that harnesses the strengths of both deep learning and traditional machine-learning algorithms. In addition, our study gives an introduction to using the Chi-Square test and Sequential Feature Selection (SFS) for feature selection and then we take the Shap and Explainable Boosting Model (EBM) for model interpretation. On top of that, we have provided a semi-supervised learning method to assess our model against unlabeled data, which is more suitable in cases where labeled data is scarce while unlabeled data is abundant such as deployments in real-world scenarios. This blended approach can maximize the use of the huge unlabeled data to boost the performance of the churn prediction model.

A two-step process was employed, beginning with long short-term memory (LSTM) and gated recurrent unit (GRU) networks to extract useful information from sequential behavioral data. The outputs from these deep learning models were then combined with original features as inputs to a Light Gradient Boosting Machine (LightGBM) classifier. This hybrid approach integrated deep and traditional machine learning techniques in a novel way to more comprehensively capture factors influencing churn. To get customer-specific information, the dataset is aggregated. Also, several algorithms were experimented with including Random Forest, LightGBM, CatBoost, Logistic Regression, LSTM, GRU, RNN, MLP Explainable boosting machine and XGBoost tree. To ready the features for the machine learning algorithms, we implemented feature engineering, transformation and selection procedures. The Chi-square test and Sequential Feature Selection (SFS) method were instrumental in identifying and retaining the most informative features. The unbalanced challenge, in which the class of churn customers was very small concerning the class of active customers, was one of the issues given by the Udacity dataset. To address the imbalance issue, the SMOTE technique was employed. We selected the optimal model by assessing its performance using a variety of evaluation measures. After selecting a model based on good accuracy and other evaluation metrics, hyper-tuning techniques with cross-validation were used to increase the accuracy of the selected model and avoid overfitting problems. Additionally, the model was utilized to create a semi-supervised model that can simulate real-world scenarios. Shap [7] was utilized to obtain local explanations for predictions made by the black-box model and EBM [8] also was used for model interpretation.

The subsequent sections of this paper are structured as follows: The second (II) section offers a comprehensive overview of related studies in this particular domain. Throughout the literature review, the focus is not only on contextualizing the current study within existing research but also on elucidating the specific contributions that distinguish this journal paper from previous works in the field. The third (III) section discusses the utilized dataset and the steps of data processing as well as feature engineering and imbalance checks. The study's findings and discussion were detailed in the fourth (IV) section, followed by the fifth (V) section which contains the conclusion and discussion.

II. LITERATURE REVIEW

Accurately predicting customer churn has garnered significant attention due to its profound impact on business profitability and customer retention strategies. Numerous studies have explored various machine learning and deep learning approaches to tackle this challenge. However, effectively capturing sequential patterns and temporal dependencies in customer data has emerged as a crucial aspect as customer behaviors and interactions with a service often exhibit temporal relationships that can indicate impending churn.

Deep learning architectures like Long Short-Term Memory (LSTM) and Convolutional Neural Networks (CNNs) have

demonstrated their effectiveness in modeling sequential information. Kwon et al. [13] leveraged an LSTM model to extract intricate time-series patterns from customer lifelog data and text messages, achieving an impressive F1-score of 89% for predicting customer attrition in the digital healthcare domain. While their approach effectively applied topic modeling and recurrent neural networks, it faced challenges in determining the optimal number of topics without prior knowledge, which could impact model accuracy. Additionally, the model's performance may be affected by the quality and completeness of the lifelog data as missing or noisy data can introduce biases and inaccuracies. Furthermore, the generalizability of their model to different types of digital healthcare apps or domains remains unexplored as the features and patterns in the data may vary significantly across different contexts. Similarly, de Caigny et al. [18] introduced a strategy aimed at enhancing churn prediction models by leveraging textual data from a European financial services provider. Their study highlighted the superiority of Convolutional Neural Networks (CNNs) over existing methods in mining sequential patterns from textual content, demonstrating improved performance through rigorous 5*3 cross-validation and metrics like AUC and Top Decile List (TDL). However, their study focused solely on CNNs as the deep learning architecture for processing textual data. While CNNs are well-suited for text classification tasks, other architectures such as recurrent neural networks (RNNs) or attention-based models may offer alternative perspectives or potentially better performance in certain scenarios. Additionally, the study did not explore the interpretability and explainability aspects of the CNN-based models, which can be crucial for practical applications and regulatory compliance in industries like finance. Furthermore, the study did not provide a comprehensive framework or guidelines for practitioners on how to effectively integrate and manage textual data sources within their existing customer relationship management (CRM) systems or data pipelines. Zorić [19] presented an illuminating case study focused on predicting customer churn within the banking industry. Leveraging a neural network within the Alyuda NeuroIntelligence software package, the researchers meticulously evaluated model performance using metrics such as CCR (correlation coefficient) and MSE, providing valuable insights into the realm of churn prediction. While neural networks have proven effective for customer churn prediction tasks, incorporating additional machine learning algorithms such as decision trees, random forests or ensemble methods could provide a more comprehensive perspective. Comparative analyses across multiple techniques would enable a deeper understanding of their relative strengths and weaknesses, facilitating the selection of the most suitable approach for a given problem context. Another potential area for improvement lies in the interpretability and explainability of the neural network models. While neural networks excel at capturing complex, non-linear patterns, they often lack transparency, making it challenging to discern the rationale behind their predictions.

While deep learning excels at capturing sequential patterns, traditional machine learning algorithms like decision trees, random forests and ensemble methods have proven effective in handling static customer attributes and features. In the domain of telecom churn prediction, Ahmad et al. [9] developed a comprehensive approach for telecom providers, incorporating four distinct algorithms: Decision Tree, Random Forest, Gradient Boosted Machine Tree (GBM) and Extreme Gradient Boosting (XGBOOST). The XGBOOST algorithm demonstrated superior performance, achieving an AUC of 93%. The dataset was divided into a 70% training and a 30% testing split with a crucial 10-fold cross-validation applied to refine hyperparameter settings. However, the study did not explore the interpretability of the models or the impact of different feature selection techniques on performance. Similarly, Jamjoom [10] delved into data mining techniques to predict client churn within insurance businesses, utilizing knowledge extraction. Their research unveiled the effectiveness of a 50:50 train-test distribution for logistic regression, while neural networks thrived with a 70:30 distribution. The decision tree models performed best with a 50:50 training/test split, achieving an AUC of 0.73. The study explored clustering techniques like k-means for identifying homogeneous customer segments but more advanced algorithms like DBSCAN have shown better performance in recent customer segmentation studies. Additionally, the study did not explicitly discuss challenges or limitations related to data availability, data quality or data preprocessing, which could impact the accuracy and reliability of the results. Li et al. [11] conducted a study focused on predicting customer churn within the cable TV industry, leveraging key indicators encompassing customer watching intensity, consumption patterns, payment behaviors and preferences. This study emphasized the significance of customer loyalty and adeptly employed the bell-shaped price curve to discern customer habits. Logistic regression emerged as the preferred tool for prediction with rigorous model performance evaluation encompassing key metrics such as AUC (62%), F1-score (58.5%) and accuracy (62.2%). While the study provided valuable insights, a potential limitation was the focus on a single cable provider which may limit the unreliability of the findings across the broadcast industry. Additionally, the reliance on behavioral data alone could overlook other factors influencing churn such as customer satisfaction and perceived value, which could be explored in future research. Shifting focus, Colot et al. [12] delve into the importance of harnessing multiple fine-grained data sources for detecting churn in telephone companies. They address the challenges associated with redundancy among extracted features. The proposed Essence Random Forest algorithm is introduced to handle such redundancy and is compared with traditional Random Forest and Extremely Randomized Trees. The research analyzes data from a European telephone

company, highlighting the potential of geospatial mobility data as a long-term alternative to the communication network. Additionally, it demonstrates the complementary value of fine-grained data for short-term churn detection. Essence Random Forest outperforms existing methods, offering enhanced classification performance of 62.9 % and faster convergence to stable results. The paper focused on the value of multiple internal fine-grained data sources for churn detection within telephone companies and assessed the value of an Essence Random Forest to handle redundancy. Still, it did not extensively discuss potential biases or limitations in the data collection process or the unreliability of the findings beyond the specific context of a European telephone company. Continuing with the exploration, Kozak et al. [14] undertook a venture to evaluate the influence of a goal-driven computational intelligence algorithm in the field of customer churn management. Their investigation focused on a thorough assessment of the effectiveness of the proposed balanced goal function of the ACDT algorithm. Various confusion metrics, including precision(0.8559), recall(0.9655) and accuracy(0.7832) were employed as crucial benchmarks for evaluating performance. While the research successfully applied various metrics like precision, recall and accuracy for performance evaluation, it did not address the challenge of connecting machine learning and artificial intelligence algorithms with decision-making tools like simulation which could enhance the practical application and adaptability of the proposed solution. In a comparative analysis, Abdel-Fattah et al. [15] conducted a comprehensive evaluation, rigorously scrutinizing the accuracy, model training and assessment of Apache Spark ML and MLlib packages. Their discerning findings underscored the prowess of MLlib, particularly its RDD-based API, which excelled in terms of training time. Conversely, ML, leveraging its DataFrame-based API, demonstrated superior performance concerning testing time and accuracy. These insights bear significant implications for businesses engaged in churn prediction, emphasizing the utility of a decision tree in conjunction with cross-validation and accuracy metrics for robust model evaluation. The accuracy value of the ML package model was 0.79 and for the MLlib package model, the accuracy was 0.73. The paper focused solely on the decision tree algorithm for churn prediction. While decision trees are widely used and interpretable, other algorithms such as ensemble methods (e.g., random forests, gradient boosting) or neural networks may offer improved predictive performance, particularly for complex and high-dimensional datasets. Future research could extend the comparative analysis to include a broader range of algorithms and evaluate their relative merits across different types of datasets and problem domains.

Recognizing the complementary strengths of deep learning and traditional machine learning, Vo et al. [16] proposed a multi-stacking ensemble approach for churn prediction in the telecommunications domain. Their innovative model combined techniques like XGBoost, Random Forest, Gaussian Naive Bayes and Logistic Regression, leveraging client call logs and customer segmentation based on profiles and personalities. SHAP and SHAP-MRMR+ were employed for interpretability, while performance evaluation centered on the AUC metric, achieving a value of 0.7362 after hyperparameter tuning. While this study highlighted the potential benefits of integrating sequential patterns and static features for improved performance and interpretability, its generalizability across industries like banking or e-commerce remains unexplored. Different sectors may exhibit distinct customer behavior patterns, data characteristics, and churn drivers, potentially necessitating adaptations to the model architecture or feature engineering processes.

Several alternative methods for handling sequential patterns have also been explored in the literature. Ahmed et al. [17] introduced the Hybrid Firefly with PSO (HFFPSO) algorithm, designed for classification within the telecom industry, particularly for large churn datasets. Their strategic approach involved deploying Spark 2.0 within a Hadoop architecture to enhance accuracy and processing speed. While the HFFPSO algorithm showed promising results, the authors noted that traditional FFA and PSO algorithms faced issues with handling highly sparse data and computational complexity, leading to inaccurate predictions for churn datasets. Additionally, the computational complexity of HFFPSO increases with dataset size, which can be a bottleneck for real-time churn prediction in big data environments.

Similar works conducted in the past using the same dataset have also been explored. Dias et al. [20] predicted customer churn with Spark for the same dataset (i.e., 12GB). They implemented a single model to predict when customers churn, either downgrading from the premium to the free tier or canceling their accounts. With this single model, they achieved an F1 score of approximately 0.80 or 80 % using the logistic regression classifier algorithm. Li et al. [21] also predicted customer churn with Spark for the same dataset but in its large variant (i.e., 12GB). They implemented their entire machine learning pipeline via Amazon AWS, incorporating a Naive predictor, logistic regression, random forest and gradient-boosted tree provided by Spark to evaluate which model yielded the best results. In the initial evaluation, the GBT classifier showed the best performance compared to the rest. After hyperparameter tuning, they managed to achieve an accuracy of approximately 0.8387 or 83% and an F1 score of 0.8229 with the GBT classifier. Sharaf et al. [22] conducted a binary classification project using a substantial dataset provided by Udacity, simulating Sparkify user activity logs. The 12GB dataset necessitated the use of Apache Spark and PySpark, making local analysis unfeasible. To address this, an Elastic MapReduce (EMR) cluster was deployed on the AWS cloud to handle extensive data processing tasks. The findings revealed that the Random Forest algorithm outperformed other models, achieving the highest accuracy of 0.875 and an F1 Score of 0.861 among the evaluated models.

In most reviewed studies, deep neural network models such as LSTM, CNN and neural networks were commonly used without the inclusion of conventional gradient-based or glass-box models and vice versa. This study contributes to filling the literature gap by utilizing not only deep learning models but also conventional models like Catboost, XGBoost and logistic regression.

Furthermore, this study proposes a novel hybrid machine-learning model that combines the strengths of deep learning and traditional machine-learning algorithms. The hybrid model initially trains an LSTM and GRU network to capture temporal patterns and dependencies in sequential customer data. Subsequently, it utilizes the predictions from the trained LSTM+GRU in conjunction with the original features as inputs to a LightGBM classifier. This integration of sequential features from the LSTM+GRU enables the LightGBM to leverage both sequential and static patterns for classification. This hybrid approach enhances the overall versatility of the predictive model, contributing to a more holistic comprehension of customer churn prediction.

While the reviewed literature showcases various approaches for customer churn prediction, a common theme emerges: the importance of effectively capturing sequential patterns and temporal dependencies in customer data, which can be achieved through deep learning architectures like LSTMs and CNNs [13], [18]. However, these approaches often focus solely on sequential information and may overlook the complementary value of static customer attributes and features [9], [10], [11], [12].

To address this gap, the proposed hybrid model in this study aims to synergize the strengths of deep learning for modeling sequential patterns with those of traditional machine learning algorithms for handling static features [16]. This approach aligns with the insights from the literature, which highlight the potential benefits of combining multiple modeling techniques and data sources to comprehensively understand customer behavior and characteristics [16], [17].

In addition to model development, this research contributes to the methodology by introducing the use of the Chi-Square test to identify relevant features and eliminate outliers that could potentially impact the proposed ML pipeline's performance. The incorporation of this statistical test offers a robust method for feature selection, ensuring the predictive model focuses on the most pertinent variables within the proposed ML pipeline's context. Additionally, the Sequential Feature Selector (SFS) is employed to scrutinize the relationship between features and labels, further validating the feature selection procedure through the Chi-Square Test. This meticulous approach enhances the accuracy and interpretability of the proposed ML pipeline.

Moreover, the paper addresses the critical aspect of model interpretation and explanation by incorporating the Shap and Explainable Boosting Model (EBM). These techniques provide insights into the internal workings of the predictive model, ensuring transparency and understanding of how various features contribute to the model's predictions. The emphasis on interpretability enhances the practical applicability of the proposed ML pipeline, rendering it more accessible and trustworthy for decision-makers.

The second issue we have dealt with is scenarios with limited labeled data but massive amounts of unlabeled data, like real-world deployment. We have, subsequently, made use of a semi-supervised learning method to solve this particular issue. This method employs a self-training technique that is used together with a highly tuned version of the Light-GBM model for semi-supervised learning. This allows the model to learn from both labeled and unlabeled data, which extends its ability to learn from data and improves its performance in different contexts.

The implications of this paper are profound for corporations and businesses, offering valuable insights into anticipating consumer patterns, restructuring marketing tactics, foreseeing market trends and enhancing overall customer satisfaction through the proposed ML pipeline. By bridging the gap between deep learning and conventional models and incorporating advanced feature selection techniques, the proposed ML pipeline presents a more nuanced and effective approach to predicting customer churn in the context of a streaming service. The model is made more accessible and trustworthy for decision-makers, ensuring that predictions can be understood and trusted in real-world scenarios.

III. METHODOLOGY

The study encompasses several sections as depicted in Figure 1, which presents a comprehensive pipeline of the entire process, starting from data preparation to model evaluation. In the initial section, the dataset undergoes preparation, including formatting and cleansing. Feature engineering, feature selection and correlation checking are carried out with additional attention given to data balancing in case of imbalances.

Afterward, the processed dataset is divided into train and test sets for model building. The models are evaluated using various metrics and the model that demonstrates good accuracy, f1 score, tpr, fpr and AUC is selected. Hyperparameter tuning is then performed.

Next, the model's performance is assessed in a scenario more closely resembling real-world conditions, incorporating semi-supervised techniques alongside a hyper-tuned model. Finally, techniques such as Shapley values and Explainable Boosting Machines are employed to provide interpretability and explainability.

A. DATASET INFORMATION

The Customer log dataset [6] is a 12.5 GB JSON file and it contains 18 columns and 26,259,199 records. The investigation was conducted to determine the type of data and the nature of the columns. There are 12 string columns and 6 numeric columns, which may also contain null or NaN values. The columns include userId, artist, auth, firstName, gender, itemInSession, lastName, length, level, location, method, page, registration, sessionId, song,

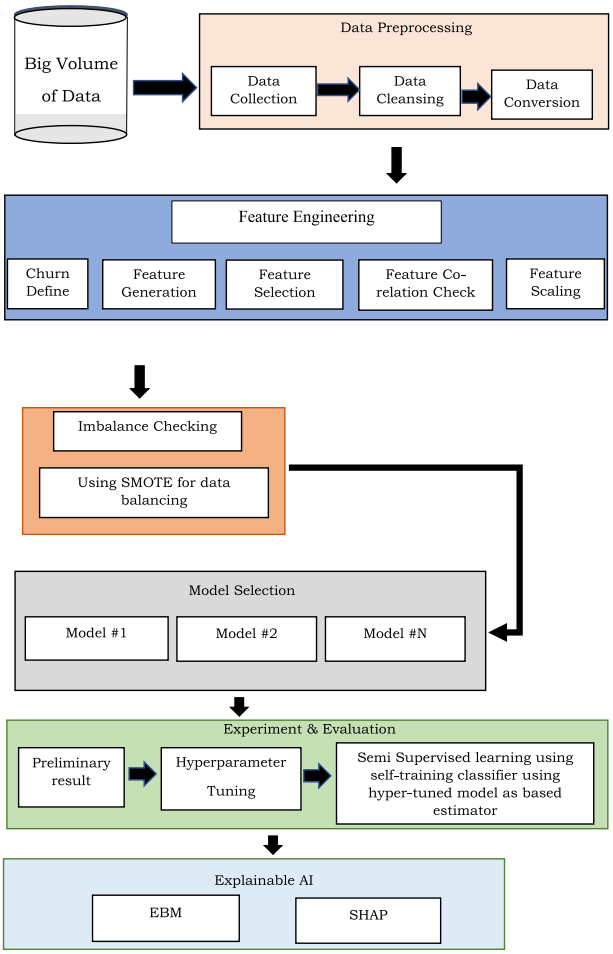

FIGURE 1. Comprehensive pipeline for model development and evaluation.

status, ts and userAgent. As evident from the column names, the dataset contains various user-related information, such as user identifiers, demographic details (firstName, lastName, gender), interaction details (artist, song, length, itemInSession, sessionId, registration, lastinteraction) and technical details (userAgent, method, page, location, status, level, auth). At the outset of our study, the number of unique customers in the dataset was unknown. To determine this, we conducted data preprocessing.

B. DATA PREPROCESSING

Data preprocessing [23] is the procedure for preparing raw data for analysis by cleaning, converting and arranging it into a format that machine learning algorithms can easily understand. Data preprocessing was conducted using Apache Spark [24].

1) CLEANSE AND VALIDATE DATA

The dataset was checked for columns containing null values, revealing 9 such columns. Correlation analysis found that entries with missing first names also had missing values in the location, registration and user agent columns. No instances were found where the first name is null but gender is not or

TABLE 1. Example of records from the Customer Log dataset without preprocessing.

Column/Feature	Row 1	Row 2
artist	Popol Vuh	Los Bunkers
auth	Logged In	Logged In
firstName	Shlok	Vianney
gender	M	F
itemInSession	278	9
lastName	Johnson	Miller
length	524.32934	238.39302
level	paid	paid
location	Dallas-Fort Worth-Arlington, TX	San Francisco-Oakland-Hayward, CA
method	PUT	PUT
page	NextSong	NextSong
registration	15337345410000	1537500318000
sessionid	22683	20836
song	Ich mache einen Spiegel - Dream Part 4	MiÃÂⓒntele
status	200	200
ts	1538352001000	1538352002000
userAgent	Mozilla/5.0 (Windows NT 6.1; WOW64)....	Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9.4...
userId	1749042	1563081

where the first name is null but the last name is not, suggesting missing values patterns. Records containing null values in userID and firstName were removed.

2) DATA CONVERSION

The timestamp and registration date columns needed to be converted to the actual date format. The timestamp was divided by 1000 and converted to month and date columns, for example, October 1st, 2018. The location column was simplified to improve comprehension.

3) CATEGORICAL VALUE TRANSFORMATION

Categorical values like gender (“Male”, “Female”) and level (“Free”, “Paid”) were converted to numerical representations using one-hot encoding [25]. For example, “Male” was encoded as 0 and “Female” as 1. This allows machine learning models to effectively learn from categorical data.

The data preprocessing steps cleansed the data by removing null values, converting timestamps and dates, simplifying locations and transforming categorical values for machine learnability [26]. This prepared the raw data for further analysis.

C. FEATURE ENGINEERING

Before creating and selecting features for the model, the churn label has been defined and added to the existing dataset. In the context of our study, “churn” refers to the phenomenon of customers discontinuing their use of a service or product. To identify churners, we used an SQL query in Spark to create a churn label for each user. If a user clicked on the “Cancel Confirmation” button, they were labeled as a churner (CHURN = 1). Otherwise, they were labeled as

non-churners (CHURN = 0). The pseudo-code for the query is as follows.

Algorithm 1 Churn Definition

```
if USER CLICKED "Cancel Confirmation" then
    CHURN ← 1
else
    CHURN ← 0
end if
```

After defining the churn label for each user, label distribution looks like Table 2:

TABLE 2. Label distribution.

	Number of Users
Label 0	17274
Label 1	5003

From Table 2, it is observed that the dataset contains a total of 22,277 distinct/unique users, out of which around five thousand users are identified as potential churners who could potentially leave the business.

1) FEATURE GENERATION / CREATION

Feature creation involves the process of converting raw data into a set of features or variables that can be utilized for training a machine learning model [27].13 features have been created from monthly data to improve the efficiency and performance of the model. These features are: 1. Avg thumb-up per user: the average number of thumb-ups received by each user in a given month. 2. Avg thumb down per user: the average number of thumb-downs received by each user in a given month.3.Gender: representation of the gender of each user. 4. Avg downgrade per user: the average number of downgrades received by each user in a given month. 5. Avg upgrade per user: the average number of upgrades received by each user in a given month. 6. Avg error per user: the average number of errors experienced by each user in a given month. 7. Avg session per user: the average number of sessions for each user in a given month. 8. Monthly session duration: the total duration of all 12.Sessions for each user in a given month. 9. Avg listen time per user: the average listen time for each user in a given month.10. level: representation of the level of each user (free or paid).11.Ad interruption: the number of times an advertisement has interrupted the user's listening experience in a given month.12.Offline listen: If the user can listen offline. 13. Registered days: the number of days the user has been registered for.

D. FEATURE SELECTION

Feature selection is a machine learning technique that chooses relevant characteristics from a large feature set to build a model [28]. Its purpose is to reduce data dimensionality and remove unnecessary or repetitive features, enhancing model accuracy and efficiency. In this paper, the Chi-Square test was used to identify relevant features and eliminate outliers that could impact model performance.

TABLE 3. Randomly selected 2 rows with 6 features after processing.

Avg Songs	Avg Likes	Avg Dislikes	Session Duration	Listen Time	Level
658.5	53.0	33.0	11404.06	658.5	Free
219.0	25.0	8.0	9026.01	219.0	Paid

1) CHI SQUARE TEST

The statistical method known as the Chi-square test is utilized to determine whether two categorical variables are significantly correlated or not [29]. In various fields including biology and social sciences, this test is widely used to assess the hypothesis that two variables are not dependent on each other. To evaluate the correlation between the target label and each feature, the chi-square test can also be applied [29]. If a high association is found, it can indicate that the feature is a good predictor of the target variable and therefore could be a valuable input in a machine learning model.

The test statistic χ^2 is calculated using:

$$\chi^2 = \sum_{i=1}^{k} \sum_{j=1}^{n} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \tag{1}$$

where k denotes the number of rows, n represents the number of columns, O_{ij} stands for the observed frequency and E_{ij} signifies the expected frequency under the assumption of independence as per the null hypothesis [29].

The p-value is calculated from the Chi-Square distribution as:

$$p = P(\chi^2 > \chi^2_{observed} | \nu) \tag{2}$$

where $\chi^2_{observed}$ is the value computed from the data and ν is the degrees of freedom.

The degrees of freedom ν for a Chi-Square test with k rows and n columns is given by:

$$\nu = (k - 1)(n - 1) \tag{3}$$

So for the "Registered day" test in the table with 314 rows and 1 column:

$$\nu = (314 - 1)(1 - 1) = 313 \tag{4}$$

Table 4 shows significant features with low P values indicating a strong relationship with the response variable. "Registered day," "Avg listen time per user," and "Avg session per user" have particularly low P values, suggesting a high correlation. Conversely, "Gender" and "Monthly session duration" exhibit higher P values, indicating a weaker correlation. The Chi-Square Value for "Registered day" is 4624.118, indicating a strong association. Similarly, "Avg thumb up per user", "Avg thumb down per user," "Avg listen time per user," and "Avg session per user" also demonstrate significant associations. However, "Gender," "Avg upgrade per user," "Avg error per user," "Level," "Ad interruption," and "Offline listen" show weaker associations. The Degree

TABLE 4. Pearson's CHI-SQUARE independence test for features against label column.

Features	P value	Chi Square Value (χ^2)	Degree of freedom
Registered day	0.02	4624.009	313
Avg thumb up per user	0.90	406.465	445
Monthly session duration	0.433	21994.667	21960
Avg thumb down per user	0.022	140.601	109
Avg listen time per user	9.70*e-16	6045.661	5194
Gender	0.211	1.563	1
Avg downgrade per user	3.089*e-71	694.250	149
Avg upgrade per user	0.159	35.373	28
Avg error per user	0.039	28.533	17
Avg session per user	1.307*e-286	1892.527	173
Level	1.094*e-13	55.189	1
Ad interruption	1.02	53.534	1
Offline listen	4.291	62.721	1

of Freedom for "Registered day" and other features varies, reflecting the number of independent observations used in the statistical tests. Considering the Degree of Freedom, P value and Chi-Square Value, 7 or 8 features could be selected as primary ones among the 13 analyzed for models. Selected features are average thumb up, average upgrade, average error, average sessions, monthly sessions, avg listen time, registered days and average downgrade per user.

2) FEATURE CORRELATION VALIDATION

The Sequential Feature Selector (SFS) [30] was employed to examine the relationship between features and labels and validate the feature selection procedure using the Chi-Square Test. SFS is a technique that selects features using a greedy search algorithm based on a predefined evaluation metric [31]. It starts with an empty set of features and iteratively adds the feature with the greatest impact on the evaluation metric until no further improvements can be made [31]. Let $X = x_1, x_2, \ldots, x_n$ represent the set of all features and $Y = y_1, y_2, \ldots, y_m$ represent the set of class labels. SFS works as follows:

$$S_0 = \emptyset \; S_{k+1} = S_k \cup \arg\max_{x_i \notin S_k} J(S_k \cup x_i), \quad k = 0, 1, \ldots, n-1 \tag{5}$$

where $J(S)$ is the evaluation criteria (e.g. classification accuracy) for the feature subset S. This helps identify the best subset of features for a model. Forward SFS was used in this study, where the best new feature is added iteratively to the selected features list. The SFS estimator for this dataset was the Extra Trees Classifier [32], a classification algorithm that uses decision trees. Extra Trees splits nodes based on a random selection of feature values, reducing the risk of overfitting. For efficiency, the forward parameter was set to True and floating to False in SFS. SFS recursively selects the best individual features to add to the subset in a greedy manner that maximizes the performance metric at each step, until reaching the desired number k of features. The dataset was partitioned into training and testing sets at a ratio of 7:3. A ten-fold cross-validation (cv=10) was then applied to the training set assessing model performance with the 'ROC AUC' metric. SFS was used to select subsets of 7 and 8 features. The features selected by SFS align closely with the results from the chi-square test. Ultimately, seven features such as Avg thumb up, average upgrade, average error, average sessions, monthly sessions, avg listen time and registered days were chosen for the churn models.

TABLE 5. Selected features using sequential forward selection (SFS).

Size	Selected Feature Subset
7	Avg thumb up, avg upgrade, avg error, avg sessions, monthly sessions, avg listen time, registered days
8	Avg thumb up, avg upgrade, avg error, avg sessions, monthly sessions, avg listen time, registered days, avg downgrade

E. FEATURE SCALING OR NORMALIZATION

Normalizing features, also called feature scaling can be helpful when the model is sensitive to variations in the scale of input features as this ensures that the model works well regardless of the different scales in the input data [33]. For feature scaling (standardization):

$$X_{\text{std}} = \frac{X - \mu}{\sigma} \tag{6}$$

where: X_{std} represents the standardized feature values, X denotes the original feature values, μ stands for the mean of the original features and σ represents the standard deviation of the original features. This centers the data and scales it to unit variance. In this study, the Standard Scaler was utilized to normalize the data after whenever the dataset was divided into training and testing sets to avoid any potential data leakage or skewed results.

F. IMBALANCED CHECKING

Due to the dataset's imbalance, models built using it may exhibit bias towards the majority class, potentially leading to inaccurate identification of the minority class [34]. To verify this assumption, a model was developed. The dataset was randomly divided into training and testing sets using a 7:3 distribution. Microsoft's Light-GBM classifier was employed to construct the model. Table 6 below showcases the classification results, including true positive rate (TPR), false positive rate (FPR), an area under the ROC curve (AUC), mean absolute error (MAE) and root mean squared error (RMSE). These metrics offer insights into the performance of the classifiers used in constructing the classification model, enabling an evaluation of the validity of our assumption regarding bias.

Although the AUC value suggests moderate performance, it is essential to recognize the limitations inherent in the classifier employed in this specific scenario. Standard

TABLE 6. Prelim light GBM result.

Classifier	AUC	TPR	FPR	MAE	RMSE
Light-GBM	0.736	0.659	0.328	0.140	0.374

machine learning algorithms often exhibit bias towards classes with a higher number of instances, disregarding minority class features as noise and predicting only the majority class [35]. Consequently, minority classes may be misclassified due to a higher likelihood of mislabeling compared to the majority classes [36]. It is essential to consider this bias when assessing classifier performance and implementing strategies like utilizing alternative algorithms or techniques to address imbalanced data [37].

1) HANDLING CLASS IMBALANCE PROBLEM USING SMOTE ALGORITHM

The previous model has a flaw due to the imbalanced distribution of classes in the dataset. To address this issue, a sampling strategy is needed as machine learning algorithms do not consider class proportions. Re-sampling methods aim to achieve a balanced class distribution by increasing instances in the minority group or reducing instances in the majority group. Random under-sampling [38] randomly removes examples from the majority class but this may result in information loss. Random over-sampling [39] replicates minority class instances but it may lead to overfitting. Cluster-Based Oversampling [40] uses K-means clustering to oversample clusters but it may overfit the training set. SMOTE [41] generates synthetic instances for the minority class and integrates them with the original data, avoiding duplicates and preserving valuable information [37]. After evaluating different re-sampling strategies, the Synthetic Minority Over-sampling Technique (SMOTE) was chosen to address the class imbalance in the dataset. SMOTE was applied to resample the classes and Table 7 presents the distribution of labels before and after applying SMOTE. Initially, the dataset had 17,274 instances of the majority class (non-churners) and 5,003 instances of the minority class (churners). After applying SMOTE, the class distribution was transformed to a balanced 50-50 distribution with 17,274 instances in each class (0 and 1). Moreover, in SVM-SMOTE [65], a different variant of SMOTE that uses a support vector machine (SVM) to identify minority instances that are close to the decision boundary, was also considered. SVM-SMOTE uses minority class instances, those positioned closest to the decision boundary, as these are the ones that would most likely be misclassified. The class distribution was also changed to a similar 50-50 distribution. 17,274 instances of each class (0 and 1) were obtained. Modeling class balancing is a frequently used technique when dealing with imbalanced datasets, in which the model is not overwhelmed by the majority class during training. By using an equal number of instances from each class, the model is forced to learn discriminative features that differentiate the two classes, without depending on the prior class probabilities. Consequently, we obtained two balanced datasets, one created using SMOTE and another using SVM-SMOTE, allowing for a comparative evaluation of these resampling techniques.

TABLE 7. Effect of SMOTE on class distribution.

	Before SMOTE	After SMOTE
Label	Number of Users	Number of Users
0	17274	17274
1	5003	17274

G. MODEL SELECTION

The model selection process involves extensively trying multiple models and making various adjustments to continuously enhance performance [42]. The goal is to find a model that can generally generalize well to unseen new data while balancing accuracy and computational efficiency. To conduct preliminary evaluation and model selection, the dataset was initially randomly split into a 60% portion for training and a 40% portion for validation and testing. However, relying on a single training/validation partition has certain limitations [43]. Having an insufficient amount of available data may result in small training and testing sets being generated which could impact the reliability of results. Utilizing a larger and more diverse validation/testing set provides a more stable and reliable estimate of key metrics like precision while exposing the learning model to a larger training dataset is more reflective of the available data that could be available for the entire learning process. Additionally, solely using a single fixed training set does not reveal the sensitivity of estimated precision or other metrics to the inclusion or exclusion of specific training samples.

During the initial preliminary modeling stage, we explored various configurations of the implemented machine-learning models. Through experimentation and evaluation, we identified the best-performing models for our dataset. We evaluated the performance of each model using metrics such as AUC, precision, recall, F1 score and accuracy. Based on these metrics, we selected the top-performing model for further analysis and evaluation.

1) HYBRID MODEL

The proposed hybrid model leverages the strengths of deep learning and traditional machine learning to enhance customer churn prediction. Initially, an LSTM [44]+GRU [45] neural network is trained on the input customer data to capture temporal patterns and dependencies over time.

The LSTM+GRU network learns the temporal dependencies in the input sequence $x_1, x_2, \ldots, x_T$ through their hidden states $h_1, h_2, \ldots, h_T$. The hidden state at each time step is computed as:

$$h_t = f_{\text{LSTM+GRU}}(x_t, h_{t-1}) \tag{7}$$

where $f_{LSTM+GRU}$ is the recurrent transition function that combines the core equations of LSTM and GRU units to model temporal relationships in the sequential data.

The predictions $\hat{y}_i$ from the LSTM+GRU network at each time step t are obtained from the corresponding hidden state h_t as:

$$\hat{y}_i = \sigma(W_h h_t + b_h) \tag{8}$$

where σ represents the sigmoid activation function, W_h is a learned weight matrix and b_h is a bias term. This maps the hidden state to a prediction between 0-1.

The proposed hybrid version nicely combines the strengths of LSTM and GRU architectures to effectively contrast its extensive temporal relationships derived from internal customer data. While LSTM excels at imaging long-term dependencies, GRU offers computational performance and excels at shooting short-term dependent features.

The LightGBM model leverages the temporal pattern information captured in the LSTM+GRU predictions $\hat{Y} = [\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T]$ along with the original input features X to make its predictions $\hat{y}$. Specifically, the LightGBM prediction function is:

$$\hat{y} = h_{\text{LightGBM}}(X, \hat{Y}) \tag{9}$$

where $h_{LightGBM}$ learns how the LSTM+GRU predictions complement the static input features to optimize classification accuracy. By incorporating representations of the temporal dependencies from LSTM+GRU, LightGBM can make more informed predictions by understanding customer behaviors over time.

Algorithm 2 Hybrid Customer Churn Prediction Model

Input:
- Customer data X_{cust} (features + labels)
- Train/test split: $X_{train}, X_{test}, y_{train}, y_{test}$

Preprocess data:
- Reshape X_{train} and X_{test} for LSTM+GRU input

Train LSTM+GRU model:
- Build LSTM+GRU network
- Train on reshaped X_{train} & y_{train}

Make predictions with LSTM+GRU model:
- $lstm_gru_train_preds = \text{LSTM+GRU}(X_{train})$
- $lstm_gru_test_preds = \text{LSTM+GRU}(X_{test})$

Reshape LSTM+GRU predictions for LightGBM:
- $X_{train_rf} = \text{Reshape}(lstm_gru_train_preds)$
- $X_{test_rf} = \text{Reshape}(lstm_gru_test_preds)$

Prepare features for LightGBM:
- $X_{train_combined} = \text{Concatenate}(X_{train_rf}, X_{train})$
- $X_{test_combined} = \text{Concatenate}(X_{test_rf}, X_{test})$

Train LightGBM model:
- Train LightGBM classifier on $X_{train_combined}$ & y_{train}

Make predictions with LightGBM model:
- $lightgbm_predictions = \text{LightGBM}(X_{test_combined})$

Evaluate predictions:
- Score model on test set using evaluation metrics

Figure 2 depicts the architecture of the proposed hybrid LSTM+GRU and LightGBM model. The original customer data is represented as a database that feeds into the deep learning model component. The deep learning component consists of an LSTM+GRU neural network designed to capture temporal dependencies in the sequential customer data. The network has an LSTM layer to capture long-term dependencies, followed by a GRU layer to capture short-term dependencies. It also includes a Batch Normalization layer for input normalization and improved stability and a Dropout layer to randomly drop inputs and reduce overfitting. Additional LSTM and GRU layers are included to further capture long-term and short-term dependencies, respectively. Finally, there is a Dense layer as the final output layer with a single neuron for binary classification (churn or not churn). The deep learning component outputs predictions representing the temporal patterns and dependencies learned from the sequential customer data. The predictions generated by the trained LSTM+GRU model are passed along with the original input features to the machine learning model component. Here, the LightGBM classifier leverages both the sequential features extracted by the LSTM+GRU network as well as the static input features. By combining representations learned from the temporal customer patterns and static data attributes, LightGBM makes the final predictions. This synergizes the abilities of deep and traditional machine learning methods. The hybrid approach effectively harnesses dependencies in customer behavior over time together with attribute-level information. This enhances the model's capability for customer churn prediction compared to using either technique in isolation.

Besides LSTM and GRU which are deep learning architectures and have proved to be able to capture sequential patterns, attention-based models, temporal convolutional networks or other sequence modeling techniques can also be the alternatives or additions to the LSTM+GRU component. These alternatives can also inform us about different sets of computational complexity, interpretability and the ability of the models to capture different kinds of sequential patterns. Such an extensive analysis of the alternatives would certainly broaden the scope of our understanding of the most efficient techniques to be used in processing sequence patterns of customer churn prediction.

To assess the effectiveness of the proposed hybrid model, we compare it to other state-of-the-art models commonly used for churn prediction. These include traditional machine learning algorithms as well as deep learning approaches that can capture sequential patterns.

LightGBM [46] is a gradient-boosting framework renowned for its efficiency in training large datasets. Similarly, XGBoost [47] stands out in the machine learning arena for its speed and high performance, often being the go-to choice in competitive environments. CatBoost [48], on the other hand, specializes in handling categorical features and handling missing data seamlessly. When dealing with sequential data, LSTM [44] and GRU [45] come into play with LSTM excelling in longer sequences and GRU offering more efficient training. The Multilayer Perceptron

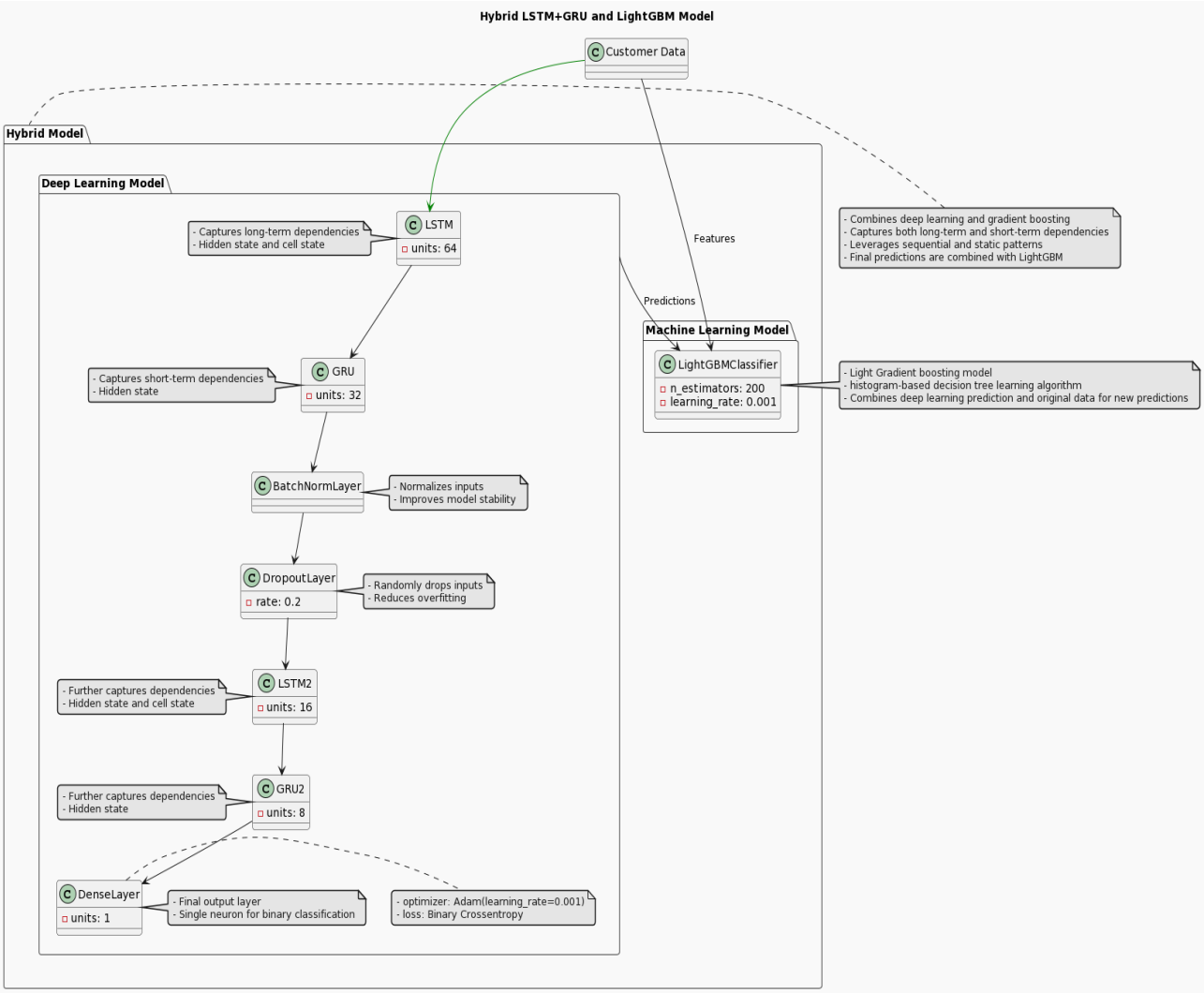

FIGURE 2. Architecture of the proposed hybrid model. The figure illustrates the flow of data through the LSTM+GRU and LightGBM components, highlighting the integration of temporal and static features for improved customer churn prediction.

[49] is an artificial neural network with multiple hidden layers, commonly employed for various machine learning tasks, including deep learning. Logistic Regression [50] is a straightforward linear classification method useful for both binary and multiclass classification. Random Forest [51], an ensemble learning technique, combines multiple decision trees to enhance predictive accuracy while reducing overfitting. Explainable Boosting Machine [52] by Microsoft stands as a model designed to offer transparency and interpretability in complex machine-learning scenarios, making it a valuable choice for applications where comprehending the model's decision-making process is vital. RNN [53] captures sequential patterns with a feedback loop, suitable for tasks where input order matters but it may struggle with long-term dependencies due to the vanishing gradient problem. The deep learning models mentioned, including LSTM, GRU, RNN and Multilayer Perceptron were implemented using Apache Spark [24] were implemented using TensorFlow [54]. Table 8 summarizes the configurations of various machine learning models that were evaluated for the task of student dropout prediction. A range of model types were considered, including ensemble methods (Random Forest, LightGBM, XGBoost, CatBoost), deep learning models (LSTM, GRU, RNN, MLP) and other algorithms (logistic regression, EBM). The hybrid model, which combines LightGBM and LSTM/GRU networks, uses a variety of hyperparameters. It trains LightGBM with 200 estimators and a learning rate of 0.001. The LSTM and GRU networks have 64 and 32 hidden units respectively with dropout and batch normalization added after the GRU layer. This model can leverage both tree-based and neural model strengths for feature extraction and sequence modeling of longitudinal data. The hybrid model's execution, encompassing both training and testing phases, required a total time of 105.65 seconds on an Intel i7-12700 CPU with 32 GB of RAM. Specifically, the deep learning component of the model consumed 105.2 seconds for training and 0.45 seconds for testing for the normal SMOTE model. Among the ensemble methods,

LightGBM and XGBoost use similar parameters with 100–200 estimators and a learning rate of 0.001. CatBoost also utilizes these settings. Random Forest was configured with a maximum depth of 3 to prevent overfitting on this dataset. The deep learning models generally employed the Adam optimizer with a learning rate of 0.001. The LSTM, RNN and MLP models had layered architectures with 64, 16, 32 and 8 hidden units.

TABLE 8. Configurations of the different models. The table provides the parameter settings for each model including the number of trees/estimators (est), learning rate (lr), number of hidden units, number of epochs, batch size (bs) and other hyperparameters. Hybrid combines LightGBM and LSTM/GRU networks. LogReg refers to logistic regression. EBM stands for Explainable Boosting Machine.

Model	Configuration
Hybrid	(LightGBM) lr=0.001, est=200, (LSTM) units=64,16 (GRU) units=32,8 adam, lr=0.001, bs=16,batch normalization & Dropout of 0.2 after (GRU) unit =32 epochs=50
LightGBM	lr=0.001, est=200
Xgboost	est=100, lr=0.001
CatBoost	est=100, lr=0.001, lr=0.001
LSTM+GRU	units=64,16 (GRU) units=32,8 adam, lr=0.001, bs=16,batch normalization & Dropout of 0.2 after (GRU) unit =32 epochs=50
MLP	units=64,16,32,8 adam, lr=0.001, bs=16,epoch=50
LogReg	default params
RandForest	criterion='log loss', max depth=3
EBM	max rounds=50, lr=0.001
RNN	units=64,16,32,8 adam, lr=0.001, bs=16,epoch=50
LSTM	units=64,16,32,8 adam, lr=0.001, bs=16,epoch=50

IV. EXPERIMENT, EVALUATION AND INTERPRETATION

A. PRELIMINARY RESULT

The model's performance was evaluated using multiple metrics including accuracy, F1 score, true positive rate (TPR), false positive rate (FPR) and area under the curve (AUC) [55]. Accuracy measures the model's correctness by calculating the ratio of correct predictions to total predictions.

$$ACC = \frac{TP + TN}{TP + FP + TN + FN} \tag{10}$$

The F1 score combines precision and recall, providing a balanced measure.

$$F_1 = 2 \times \frac{P \times R}{P + R} \tag{11}$$

where Precision (P) measures the proportion of true positive predictions among all positive predictions made by the model. It is calculated as:

$$P = \frac{TP}{TP + FP} \tag{12}$$

and Recall (R) measures the proportion of true positive predictions among all actual positive instances in the dataset. It is calculated as:

$$R = \frac{TP}{TP + FN} \tag{13}$$

A higher F1 score indicates a better balance, although accuracy can be misleading with imbalanced datasets. TPR measures the correct classification of positive instances, while FPR measures the incorrect classification of negative instances as positive.

$$TPR = \frac{TP}{TP + FN} \tag{14}$$

$$FPR = \frac{FP}{FP + TN} \tag{15}$$

AUC represents the overall performance of a binary classifier, measuring the separation between positive and negative classifications using the ROC curve. A higher AUC indicates better classification performance.

Tables 9 and 10 comprehensively compare the performance of various classification models employed in this study, using two different techniques for handling class imbalance: Normal SMOTE and SVM-SMOTE.

TABLE 9. Comparison of model performance using SMOTE for the customer log dataset.

Models	AUC	ACC	F1	TPR	FPR
Hybrid Model	**0.9157**	**0.8455**	**0.8371**	**0.7961**	**0.1053**
MLP	0.8324	0.7558	0.7439	0.7116	0.2002
RNN	0.8926	0.8120	0.8070	0.7884	0.1645
LSTM + GRU	0.8796	0.7954	0.7840	0.7450	0.1545
GRU	0.8931	0.8096	0.8022	0.7743	0.1552
LSTM	0.8844	0.7960	0.7895	0.7675	0.1756
LightGBM	0.8865	0.8134	0.7998	0.7476	0.1212
CatBoost	0.8599	0.7620	0.7287	0.6410	0.1177
XGBoost	0.8646	0.7754	0.7557	0.6969	0.1466
RandomForest	0.8354	0.7497	0.7096	0.6134	0.1148
EBM	0.7791	0.7088	0.6889	0.6467	0.2294
Logistic Regression	0.8382	0.7643	0.7617	0.7556	0.2270

TABLE 10. Comparison of model performance using SVM-SMOTE for the customer log dataset.

Models	AUC	ACC	F1	TPR	FPR
Hybrid Model	**0.9258**	**0.8583**	**0.8544**	**0.8342**	**0.1177**
MLP	0.8605	0.7879	0.7847	0.7753	0.1995
RNN	0.9099	0.8326	0.8249	0.7910	0.1260
LSTM + GRU	0.9125	0.8343	0.8235	0.7757	0.1075
GRU	0.9149	0.8378	0.8323	0.8074	0.1320
LSTM	0.9159	0.8391	0.8325	0.8019	0.1238
LightGBM	0.8834	0.8174	0.8024	0.7434	0.1089
CatBoost	0.8608	0.7892	0.7679	0.6994	0.1215
XGBoost	0.8723	0.7966	0.7688	0.6785	0.0860
RandomForest	0.8553	0.7668	0.7288	0.6285	0.0958
EBM	0.8224	0.7409	0.7247	0.6843	0.2029
Logistic Regression	0.8606	0.7919	0.7860	0.7669	0.1832

When using Normal SMOTE (Table 9), the Hybrid Model (LightGBM with LSTM+GRU) exhibits strong performance, achieving an AUC of 0.9157, accuracy of 0.8455, F1-score of 0.8371 and TPR of 0.7961. However, the model's performance is further improved with the application of

TABLE 11. Model training and testing times for SVM-SMOTE and normal SMOTE datasets.

Model Name	Train Time (s) svm-smote	Test Time (s) svm-smote	Train Time (s) smote	Test Time (s) smote
LightGBM	0.39	0.05	0.31	0.06
CatBoost	0.57	0.01	0.45	0.01
XGBoost	0.76	0.05	0.13	0.02
RandomForest	3.05	0.19	0.75	0.05
EBM	4.68	0.02	4.10	0.01
Logistic Regression	0.06	0.02	0.01	0.01
Hybrid Model	156.64	0.46	105.2	0.45
GRU Model	65.82	0.66	69.50	0.67
MLP Model	32.09	0.30	32.62	0.41
RNN Model	92.18	0.62	87.13	0.58
LSTM+GRU Model	114.01	1.03	100.67	0.94
LSTM Model	80.48	0.60	63.32	0.58

SVM-SMOTE (Table 10), where it attains an AUC of 0.9258, accuracy of 0.8583, F1-score of 0.8544, TPR of 0.8342, and FPR of 0.1177. Of the two SMOTE techniques, the Hybrid Model consistently surpasses other models in terms of overall performance metrics. While deep learning models like LSTM, GRU and RNN exhibit competitive results, the Hybrid Model surpasses them, particularly when using Normal SMOTE. LightGBM combines the gradient boosting method with LSTM and GRU temporal features extraction resulting in a model with better predictive accuracy than the standalone deep learning models. By comparison with the non-deep-learning models such as the XGBoost, CatBoost, Random Forest and Logistic Regression, the Hybrid Model consistently outperforms them across most evaluation metrics, regardless of the technique used for handling class imbalance. However, some of these models, despite their similar accuracy level; their AUC, F1-score and TPR/FPR balance are not as strong as the Hybrid Model's performance.

Table 11 compares model training and testing times between SVM-SMOTE and SMOTE respectively. The Hybrid Model (156.64s for SVM-SMOTE and 105.2s for normal SMOTE) has the longest training time because of its complex architecture but its testing time (0.46 for SVM-SMOTE and 0.45 for normal SMOTE) is relatively low. The table also identifies the differences in training time between the two methods - for instance, Logistic Regression training time is reduced from 0.06s with SVM-SMOTE to 0.01s with normal SMOTE while the testing time stays the same. This implies that normal SMOTE probably can be better in terms of computational efficiency for some of the models but the overall result still can be different.

The Sample Telecommunication dataset [57] provided by IBM and a Flight booking dataset [58] obtained from the "Ease My Trip" website are also used to compare the performance of the hybrid model against other models using the same configurations. In the IBM telecommunication dataset, the Hybrid Model, incorporating LightGBM with LSTM and GRU architectures, demonstrates outstanding performance with an AUC of 0.9611 and the second-best TPR of 0.9836. In this dataset, the Hybrid Model is employed to predict churn values. On the flight booking dataset, the Hybrid Model achieves commendable performance, securing the best AUC of 0.7756, showcasing the model's effectiveness in predicting flight seat class. The results of the IBM telecommunication and flight booking datasets are shown in Table 12.

TABLE 12. Comparison of model performance for the IBM telecommunication and Flight booking dataset.

Models	ACC	F1 Score	TPR	FPR	AUC
Hybrid Model (Telecom)	**0.8780**	**0.9005**	**0.9836**	**0.2572**	**0.9611**
MLP (Telecom)	0.8838	0.8991	0.9460	0.1731	0.9248
RNN (Telecom)	0.8982	0.9121	0.9517	0.1601	0.9375
GRU (Telecom)	0.9026	0.9135	0.9390	0.1211	0.9490
LSTM (Telecom)	0.8987	0.9109	0.9460	0.1391	0.9482
LSTM+GRU (Telecom)	0.8503	0.8786	0.9648	0.2962	0.9341
LightGBM (Telecom)	0.8824	0.9039	0.9851	0.2492	0.9457
CatBoost (Telecom)	0.8762	0.8948	0.9375	0.2022	0.9292
XGBoost (Telecom)	0.8964	0.9121	0.9570	0.1811	0.9464
RandomForest (Telecom)	0.8793	0.8975	0.9406	0.1991	0.9416
EBM (Telecom)	0.8613	0.8846	0.9468	0.2482	0.9237
Logistic Regression (Telecom)	0.8933	0.9086	0.9445	0.1721	0.9456
Hybrid Model (Flight)	**0.6677**	**0.6837**	**0.5217**	**0.0096**	**0.7756**
LightGBM (Flight)	0.6646	0.6795	0.5163	0.0077	0.7745
CatBoost (Flight)	0.6466	0.6562	0.4897	0.0066	0.7679
XGBoost (Flight)	0.6574	0.6717	0.5091	0.0147	0.7653
RandomForest (Flight)	0.6510	0.6795	0.5371	0.0972	0.7660
EBM (Flight)	0.6531	0.6893	0.5588	0.1383	0.7539
Logistic Regression (Flight)	0.5860	0.6504	0.5594	0.3552	0.6581
LSTM+GRU (Flight)	0.6697	0.6947	0.5459	0.0568	0.7673
LSTM (Flight)	0.6661	0.6855	0.5284	0.0295	0.7694
GRU (Flight)	0.6657	0.6846	0.5269	0.0272	0.7680
RNN (Flight)	0.6726	0.7001	0.5550	0.0673	0.7712
MLP (Flight)	0.6314	0.7080	0.6488	0.4069	0.6603

As with the previous dataset, the hybrid model performed pretty well on the IBM telecommunication as well as flight booking data. This validates that the proposed hybrid model is capable of accurately modeling classification problems without being too tailored to a specific domain or dataset.

The combination of the interpretable gradient boosting strategy of LightGBM with the representation learning ability of recurrent neural networks leads to a very robust and accurate classification model. The power of temporally aware neural network features coupled with LightGBM's iterative learning helps uncover subtle patterns in the time-series or sequence data which are difficult to model using only traditional algorithms or isolated deep learning architectures. Therefore, given its state-of-the-art evaluation results, the Hybrid Model (LightGBM boosted with LSTM+GRU features) would be the most suitable candidate for hyperparameter optimization and developing the best possible predictive model for this classification task.

B. HYPERPARAMETER TUNING

The Repeated Stratified Fivefold K-Fold cross-validator [56] with three-time repetition and Grid Search was utilized to find the best hyper-parameters [60] for the selected Hybrid model. Grid Search is a fundamental algorithm for hyper-parameter tuning, where the hyper-parameter domain is divided into discrete grids and all possible combinations are tested using cross-validation to calculate performance metrics [59]. The best combination of hyper-parameter values is determined based on the increase in overall average performance in cross-validation. Grid Search is comprehensive but can be slow due to the need to check every possible combination. However, it is effective in finding the best hyper-parameter values. For the LightGBM model, Grid Search was employed with a set of grid samples to find the optimal parameters. To ensure more reliable estimated performance, a repeated Stratified cross-validator was used, involving multiple repetitions of the cross-validation process and calculation of the mean result across all folds and runs. Grid Search for hyperparameter tuning was performed on an Intel i7-12700 CPU with 32 GB of RAM. It took approximately 38 minutes to complete. Table 13 presents the grid samples and the optimal parameters obtained through Grid Search.

TABLE 13. Hyperparameter tuning for LightGBM.

	Model Parameters		Regularization	
	Esti-mators	Learning Rate	Subsample	Max Depth
Grid Search	100-2000	0.001-0.1	0.3-1.0	7-15
Optimal	2000	0.01	0.3	12

For the deep learning part of the model, Keras Tuner [61] was used to optimize the hyper-parameters. Keras Tuner implements Hyperband, an algorithm for automatic hyper-parameter optimization. It builds multiple deep learning models with different hyper-parameter configurations specified using the Keras Tuner Hyper-parameters class. These include the number of units in each deep learning layer, learning rate etc. The model is trained on different hyper-parameter configurations and evaluated on validation set accuracy. The hyperband algorithm [62] early stops unpromising runs to optimize resource allocation. The best hyper-parameter set is selected based on the model that achieves the highest validation accuracy. This helps find the optimal architecture and hyper-parameters for the deep learning part of the model in an automated manner. Keras Tuner was also performed on the same device as Grid Search with a validation set of 10% and It took approximately 35 minutes to complete. Table 14 presents the parameter samples and the optimal parameters obtained through Tuner.

We divided the SMOTE applied dataset into training and test sets with a ratio of 9:1 with 90% allocated to the train set and 10% to the test set. The model was then trained using the obtained optimal parameters and subsequently assessed on the test dataset. The hyperparameter-tuned hybrid model took 137.642 sec to execute. Compared to the preliminary results of SMOTE applied dataset, there was a 4.03% increase in AUC, a 5.7% increase in Accuracy, a 6.38% increase in F1 score and a 7.48% increase in TPR from the baseline model. For FPR, it's not appropriate to calculate a percentage increase because a lower FPR is better and it has also decreased from the baseline model.

TABLE 14. Hyperparameter search space and optimal values for deep learning model.

Hyperparameter	Search Space	Optimal Value
Units in 1st LSTM layer	64-256 (step 16)	240
Units in 1st GRU layer	32-128 (step 16)	96
Units in 2nd LSTM layer	16-64 (step 16)	64
Units in 3rd GRU layer	8-64 (step 16)	8
Learning rate	0.1-1e-4	0.01

TABLE 15. Result of hyper-tuning of hybrid model for customer log dataset.

Model	Acc.	F1	TPR	FPR	AUC
Hybrid Model	0.9025	0.9009	0.8709	0.0648	0.9560

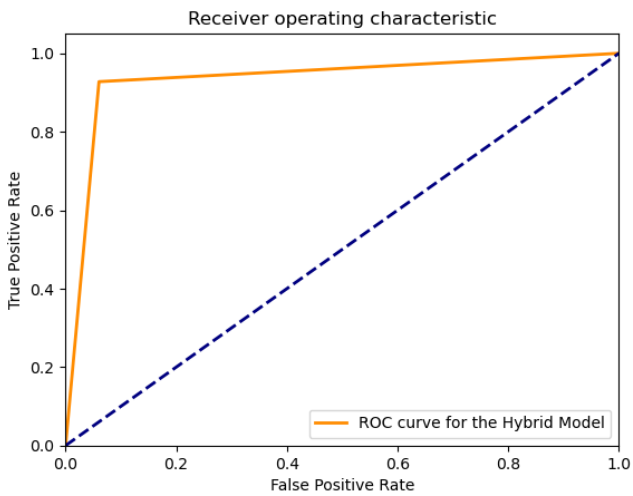

FIGURE 3. ROC Curve for Hybrid model shows that the model can identify positive cases with a high degree of accuracy while maintaining a relatively low false positive rate.

The effectiveness of the hyper-tuned model was assessed through the utilization of a Receiver Operating Characteristic (ROC) curve [63]. This curve visually represents the model's accuracy across various classification thresholds, plotting the True Positive Rate (TPR) against the False Positive Rate (FPR). A diagonal line indicates random performance, while a point at the top-right corner signifies optimal performance. The ROC curve provides a visual representation of the model's performance. In this study, the ROC curve was constructed for the Hybrid model after hyperparameter tuning. The curve indicates how well the model can predict the category of an item. In Figure 3, the ROC curve shows promising results with the curve tending toward the top-right corner, indicating the model can distinguish between positive and negative cases better than chance and that it is likely to be a good predictor of the outcome of interest.

In order to maintain neutrality and fairness, after the hybrid model, we adjusted the optimal parameters of the best three models to see if the performance of these models could be further improved as well. Hyperparameter tuning is a resource-demanding and time-consuming activity, so we used LSTM, GRU and RNN which are the models after the hybrid model with the best performance.

Table 16 portrays the main results of hyperparameter tuning for the LSTM, GRU and RNN model which demonstrated that the best configurations led to improvements in many performance metrics including accuracy, F1 score, true positive rate and AUC.

Furthermore, table 17 outlining the optimal hyperparameters for each model provides valuable insights into the architectural choices that led to the best results.

TABLE 16. Result of hyper-tuning of LSTM, GRU, and RNN models for customer log dataset.

Model	Acc	F1	TPR	FPR	AUC
LSTM	0.8295	0.8368	0.8589	0.2009	0.8990
GRU	0.8301	0.8354	0.8476	0.1880	0.9083
RNN	0.8174	0.8153	0.7924	0.1567	0.8928

TABLE 17. Optimal hyperparameters for LSTM, GRU, and RNN models.

Hyperparameter	LSTM	GRU	RNN
Units in 1st layer	144	88	72
Units in 2nd layer	76	44	108
Dense units in 1st layer	48	24	60
Dense units in 2nd layer	4	6	6
Learning rate	0.01	0.01	0.001

C. USED SEMI-SUPERVISED LEARNING TECHNIQUE TO CHECK THE MODEL AGAINST UNLABELED DATA

In many practical cases, acquiring large labeled datasets may become a fancy and time-consuming process, especially in areas like healthcare, finance and security where data labeling should be done by experts. The main difficulty for the supervised learning methods with the traditional approach is the data scarcity with labels as they need abundant labeled examples to train accurate models. As a solution to this problem and also capitalizing on the available unlabeled data, we subsequently used a semi-supervised learning approach. We particularly did this by partially unlabeling a subset of the available labeled data to form an unlabeled dataset that was later used together with the remaining labeled data for semi-supervised learning.

In semi-supervised learning, a model is trained by leveraging both labeled and unlabeled data, integrating the benefits of both supervised and unsupervised approaches [64]. Labeled data guides the model's predictions, while unlabeled data offers additional learning opportunities.

Let $X_l = (x_1, y_1), \ldots, (x_n, y_n)$ be the labeled training data where $x_i \in \mathcal{X}$ are the features and $y_i \in \mathcal{Y}$ are the labels. Let $X_u = x_{n+1}, \ldots, x_m$ be the unlabeled training data. The goal of semi-supervised learning is to learn a function $f : \mathcal{X} \to \mathcal{Y}$ by utilizing both the labeled data X_l and unlabeled data X_u.

A common approach is to formulate semi-supervised learning as an optimization problem:

$$\min_f \sum_{(x_i, y_i) \in X_l} \mathcal{L}(f(x_i), y_i) + \Omega(f) \tag{16}$$

where $\mathcal{L}(.)$ is a loss function measuring the error on labeled data (e.g. cross-entropy for classification) and $\Omega(f)$ is a regularization term encouraging the consistency of predictions on unlabeled data. For example, in self-training $\Omega(f)$ can encourage $f(x)$ to be close to the predicted label $\hat{y}$ for each $x \in X_u$.

By jointly optimizing this objective, the model f can leverage both labeled and unlabeled data to learn representations and make more accurate predictions.

In this study, a self-training technique was applied using a hyper-tuned Light-GBM model for semi-supervised learning. Self-training iteratively optimizes the following objective:

$$\min_f \sum_{(x_i, y_i) \in X_l} \mathcal{L}(f(x_i), y_i) + \sum_{x_j \in \hat{X}_u} \mathcal{L}(f(x_j), \hat{y}_j) \tag{17}$$

where $\hat{X}_u$ is the subset of unlabeled data X_u that is pseudo-labeled with predicted labels $\hat{y}_j = f(x_j)$ using the current model f. The first term optimizes for labeled data, while the second term encourages consistency between the model and its own predictions on unlabeled data.

Self-training is an iterative process where a model is first trained on the labeled data. It then makes predictions on the unlabeled data and the most confident predictions are added as "pseudo-labeled" examples to the training set. The model is re-trained on this augmented dataset containing the original labeled data combined with the pseudo-labeled unlabeled data. This process is repeated for multiple iterations, allowing the model to progressively learn from its own high-confidence predictions on unlabeled data.

This approach can achieve better performance with fewer labeled examples compared to solely relying on supervised learning [64]. To address the class imbalance in the original dataset, SMOTEENN [66], a technique that combines over & under-sampling using SMOTE and then Edited Nearest Neighbours [67] was applied. After resampling, the dataset consisted of 18,865 records and was split into a 2:8 ratio with 20% allocated to the train set and 80% to the test set. The train set comprised 3,773 rows, while the test set contained 15,092 unlabeled rows. The training dataset contained only 189 labeled rows and 3,584 unlabeled rows, which were utilized for self-training. After training and testing the model, the results are presented in Table 18.

The results presented in Table 16 demonstrate favorable performance across all evaluation metrics used. To replicate

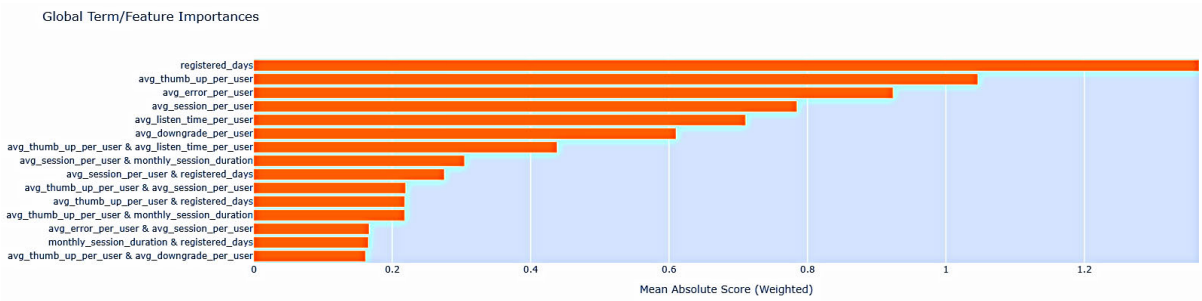

FIGURE 4. Feature importance generated by EBM.

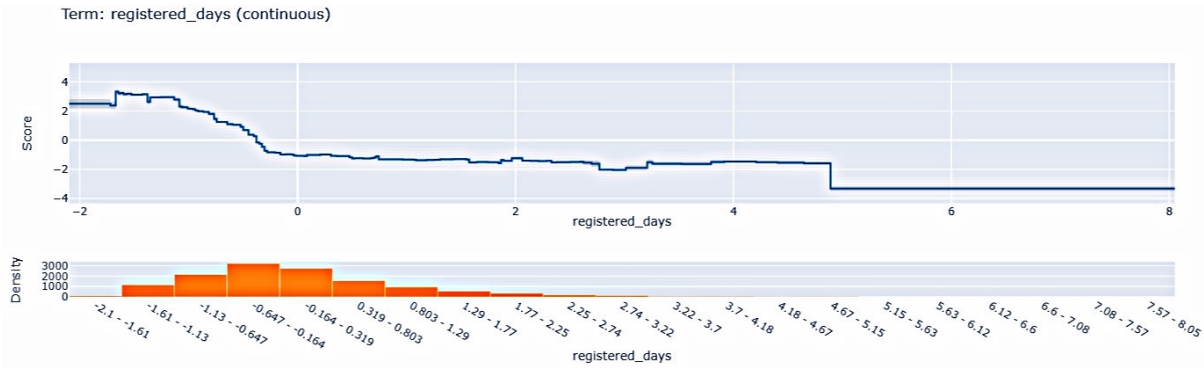

FIGURE 5. Term score plot showing the contribution of the 'registered days' feature to predictions made by the EBM model. A higher score indicates a stronger positive or negative influence on the predicted outcome.

TABLE 18. Result of self training classifier on balanced customer log dataset.

Model	Acc.	F1	TPR	FPR	AUC
Self-Training Classifier with LightGBM	0.826	0.82	0.877	0.420	0.827

real-world scenarios, an imbalanced dataset was deliberately employed. Such situations typically involve only a small portion of customers leaving, while the majority tend to stay unless significant changes occur. The imbalanced dataset consisted of 22,277 rows, which were divided into a train and test set using a 2:8 ratio (a train set of (20%) and a test set of (80%)). To tackle the data imbalance, a larger proportion of labeled data was included in the training dataset. Specifically, the training dataset comprised 4,454 rows with 2,005 labeled instances and 2,450 unlabeled instances and the test set contained 17,823 unlabeled rows. Following the training and testing of the model against the test dataset, the results can be observed in Table 19.

TABLE 19. Result of self training classifier on imbalanced customer log dataset.

Model	Acc.	F1	TPR	FPR	AUC
Self-Training Classifier with LightGBM	0.844	0.84	0.669	0.345	0.732

An F1 score of 84% was achieved, along with an AUC of 73%. Due to the nature of the imbalanced data, the TPR is understandably lower.

D. EXPLAINABLE AI

1) EBM

EBM is a glass box [52] model that strives for accuracy comparable to complex models like Random Forest and Boosted Trees. Unlike black box models, EBM supports both global and local explanations.

By employing EBM, the dataset's global explainability and feature importance were assessed, along with local explainability for individual user labels. The following section presents the global and local explainability provided by EBM. In Figure 4, EBM has listed the top feature contributions in descending order. The mean absolute contribution (score) of each word (feature or interaction) to predictions is averaged throughout the training dataset [52]. Contributions are weighted according to the number of samples in each bin as well as the sample weights (if any) [52].

Figure 5 shows the contribution (score) of the term "registered days" to predictions made by the model as well as the density of data points across different values of registered days. According to the top panel, the term score is generally higher for lower values of registered days up until around 3-4 days, after which the score decreases substantially. This suggests that having been registered for a very short period is a stronger positive indicator for this term than longer registration periods. The bottom density plot reveals that there are relatively fewer data points for very high registered day values beyond around 6-7 days. Overall, Figure 5 demonstrates that while the number of

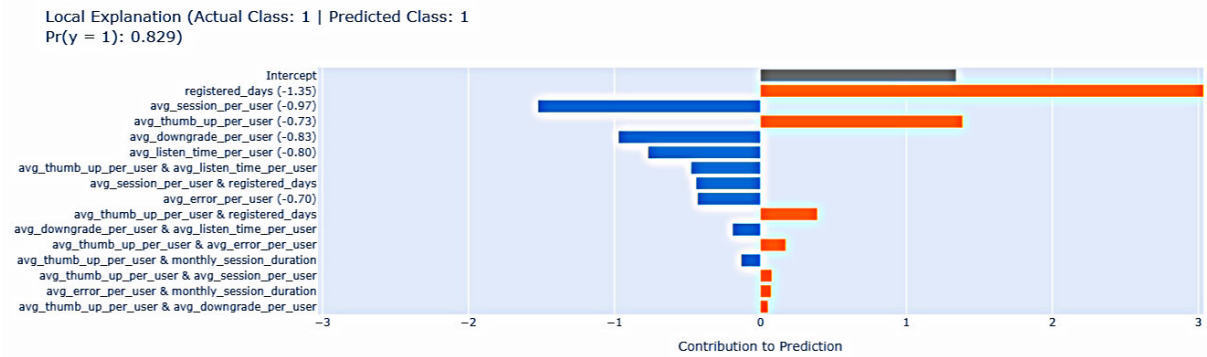

FIGURE 6. Local explanation from EBM for a sample with an actual label of 1 that was predicted as 1. The plot shows the contribution of each feature to the prediction for this specific sample.

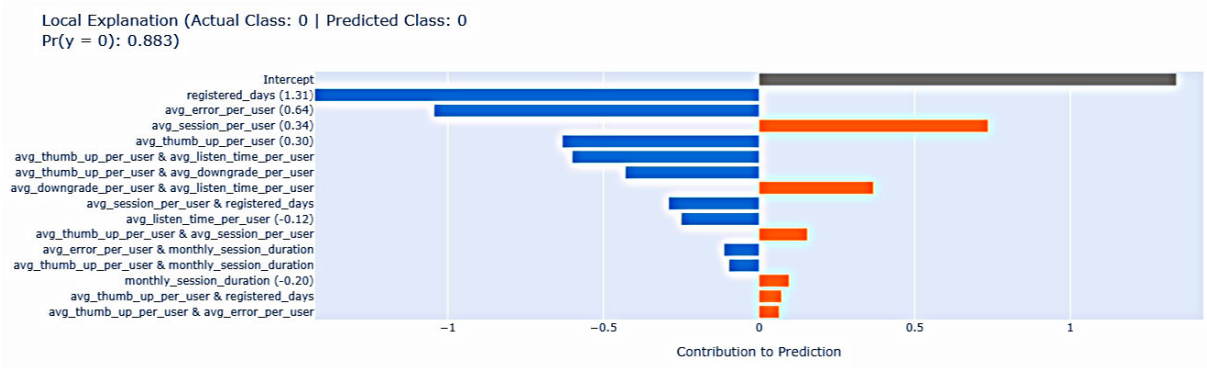

FIGURE 7. Local explanation from EBM for a sample with an actual label of 0 that was predicted as 0. The plot reveals the feature contributions to the prediction for this individual sample.

registered days influences the model's scoring for this term, the relationship is non-linear. The term score peaks for relatively new registrations under 4 days but drops off substantially after that point, regardless of how long the registration period extends.

A local explanation indicates how much each phrase contributed to the forecast for a single user with label 1 in Figure 6 [7]. The figure generated from EBM shows the local explanation for an individual's predicted class. The predicted class is 1, which means that the individual is likely to have a high term score. The local explanation shows the contributions of different features to the prediction. Figure 6 shows that the following features have the strongest positive contributions to the prediction: registered days, average session per user and average thumb up per user. The following features have the strongest negative contributions to the prediction: average downgrade per user and average error per user. Figure 6 also shows some interactions between features. For this specific instance, the interaction between average thumb up per user and average listen time per user has a positive contribution to the prediction, indicating that the combination of high values for these features contributed to the model's prediction of a high term score for this individual. Figure 7 generated from EBM shows the local explanation for an individual's predicted class. The predicted class is 0, which means that the individual is less likely to have a high term score. The local explanation shows the contributions of different features to the prediction. It shows that the following features have the strongest negative contributions to the prediction: monthly session duration and average listen time per user. Figure 7 also shows some interactions between features. For example, the interaction between average thumb up per user and average listen time per user has a positive contribution to the prediction for this individual. However, it's important to note that this is a local explanation specific to this individual's prediction and should not be generalized as a global interpretation without further analysis. This figure suggests that this specific user who has a high average number of thumbs up and a high average listen time per user is more likely to have a high term score, even if the user's monthly session duration and average listen time per user are low. Based on the feature contributions, it seems that the EBM predicted this individual as not likely to churn (class 0) because the user had a longer monthly session duration, listened to music for a longer average time per session, gave more thumbs up and had fewer errors per session. Additionally, the interactions between these features, particularly between average thumb up per user and other features, seem to further strengthen the prediction of non-churn.

2) COMPARING GLOBAL AND LOCAL EXPLANATIONS

The global feature importance plot from the EBM model demonstrates the overall contribution of individual features in the entire dataset. It shows that the features including registered days, average thumb up per user, average error per user and average session per user are the most important ones that can be used to predict the model. On the other hand, the individual EBM plots give a local explanation of how the overall importance of the variables shows up differently in specific instances. For example, in the local explanation for the non-churn sample (Actual Class: 0), while registered days has a strong positive influence aligning with its high global importance, average error per user also exerts a notable positive effect despite having lower global importance. Conversely, for the churn sample (Actual Class: 1) among the features, ones like average session per user, average thumb up per user and average downgrade per user exhibit strong negative influences. This indicates they are the risk factors for that specific instance, although their global importance scores are just moderate. These local explanations show that global feature importance is the average effect but individual predictions might be driven by subtle feature interactions and value-based influences specific to each case. Local EBM plots are an instance-level window for such particularities, which usually complement the global view. In addition, the global plot is a mixture of class-specific elements, while the local plots display universal class effects. In the case of the churned sample, registered days and average thumb up per user increase the churn risk, whereas in the case of the non-churned sample they reduce the risk. The global explanations from EBM are what present the general prominent features. The local explanations, on the other hand, unravel how the features interact and present their influences which lead to predictions for individuals. These approaches give a multi-dimensional view of interpretation.

3) SHAP EXPLAINABILITY

SHAP (Shapley Additive exPlanations) serves as a framework designed to interpret predictions made by machine learning models [7]. It employs game theory concepts to explain the predictions of any machine learning model by calculating the contribution of each feature to the prediction [7]. To assess the explainability provided by SHAP, the hyper-tuned LightGBM model with selected features available in the dataset was utilized.

In Figure 8, a SHAP summary plot is presented, showcasing the average influence of each feature on the model's output organized by the predicted class. The y-axis lists the features and the x-axis displays the average SHAP value. A higher SHAP value signifies a more substantial contribution of the feature to increasing the model's output, whereas a lower SHAP value implies a more pronounced contribution to decreasing the model's output. The plot reveals that the feature's average thumb up per user, average listen time per user and monthly session duration exert the most significant positive impact on the model's output for both classes. This implies that users who provide more thumbs up, listen to more music and have longer monthly session durations are more likely to be classified as not churned. Conversely, the feature's average thumb down per user and average error per user has the most substantial negative impact on the model's output for both classes. This indicates that users who provide more thumbs down or make more errors are less likely to be classified as not churned. The feature registered days exhibit a mixed impact on the model's output.

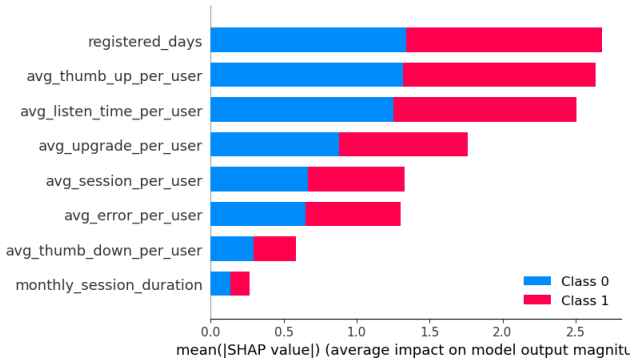

FIGURE 8. Average feature impact on model output for each predicted class. Higher SHAP values indicate a positive contribution to increasing the model output, while lower values suggest a contribution to decreasing the model output.

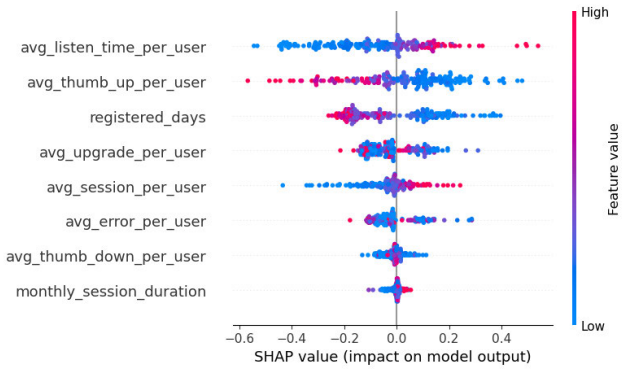

FIGURE 9. SHAP values of features impacting model output. Higher SHAP values indicate a positive contribution to increasing the log odds, while lower values suggest a contribution to decreasing the log odds.

Figure 9 is another SHAP summary plot showing the average impact on the magnitude of the model output for each feature, split by the predicted class. The plot reveals that features such as average listen time per user and average thumbs up per user have the greatest positive impact on the magnitude of the model's predictions for both classes. In contrast, the average error per user negatively impacts the output magnitude. Features like registered days exhibit a mixed impact depending on the class. Both average upgrades and sessions add modest upward nudges, while average errors and average thumb downs per user tug the prediction slightly downwards. Interestingly, monthly session duration takes the backseat in terms of influence. This visual analysis of feature impacts not only sheds light on the key players but also offers a glimpse into their relative importance and potential hidden relationships within the data.

4) VALIDATING EXPLANATIONS

The EBM and SHAP approaches offer different perspectives regarding global and local interpretations. As a result, by comparing their findings and cross-validating the scores of variables and the effect of each on the overall model, we can validate the results. Globally, four attributes are implicated in the model by EBM's feature importance plot (Figure 4) and SHAP's summary plots (Figures 8 and 9), which are average thumb up per user, average listen time per user, average session per user and registered days that are responsible for model predictions and output magnitudes. The alignment of the two methodologies here ensures that the key factors identified as important are not forgotten. As a whole, the agreement of EBM with SHAP's global explanations fosters a degree of belief in the main determinants and the local explanations are just like having a multi-dimensional view of how the features are interacting and contributing to the personal predictions. Alongside making certain that the explanatory capabilities of Explainable Boosting Machine (EBM) and SHapley additive exPlanations (SHAP) methods, building trust in the insights derived is also a vital task. Several approaches can also be employed:

1) Feature perturbation: By the systematic modification or perturbation of particular features and observing the consequent changes in the model performance and explanation, we can verify if the model explanations follow the expected behavior. Discrepancies may reflect the problems that can persist in the given explanations.
2) Cross-Validation: From EBM's global and SHAP's explanations we can apply cross-validation strategies. Their agreement on identifying key features like registered days, average thumb up/down per user, average listen time per user and average session per user as highly important (Figures 4, 8 and 9) reinforces the validity of these insights.
3) Domain Expert Evaluation: Engaging domain experts to review the explanations can provide valuable feedback. If the explanations resonate with their domain knowledge and align with expectations, it increases confidence in their validity.
4) Consistency Checks: Bringing explanations together from different samples or subsets of the data will make it possible to detect disagreements or contradictions. For instance, the local explanation plots (Figures 6 and 7) demonstrate how the same features can exhibit varying influences for individual instances, offering a multi-dimensional perspective.
5) Ensemble of Explanation Techniques: Through interrelated explanation techniques and cross-checking their insights, the reliability of the results can be improved. The congruence of EBM and SHAP's global explanations on feature importances increases the reliability of the extracted feature importances.

Via contrasting global and local interpretations of EBM with SHAP and its validation methods such as feature perturbation and ensemble of techniques, we can gain a more comprehensive knowledge of the models, build more confidence in the derived insights and identify directions for further exploration.

V. CONCLUSION AND DISCUSSION

The goal of this study was to accurately predict customer churn in the streaming service market to help companies retain more customers. Several machine learning models were evaluated on their ability to classify customers as churners or non-churners based on customer profile and usage data. The proposed hybrid machine learning model achieved the highest performance with an area under the ROC curve of 95.60% and F1 score of 90.09% when evaluated on a held-out test set. This model significantly outperformed other commonly used algorithms such as random forests, neural networks and gradient-boosting machines. Additionally, a semi-supervised technique called self-training was able to achieve promising results with only a small amount of labeled data, demonstrating the potential for predictive models even when labels are scarce. Model interpretation techniques including SHAP and Explainable Boosting Machines provided insight into how and why the models made predictions. This added transparency helps ensure the models can be appropriately used and trusted to inform business decisions. This study demonstrates the effectiveness of various machine learning techniques for customer churn prediction using streaming service data. The results and findings offer several insights that could guide future work. The hybrid model combining LSTM+GRU and LightGBM achieved the best performance indicating the advantages of incorporating deep learning and tree-based algorithms. The sequential modeling ability of LSTM+GRU complemented LightGBM's efficient handling of large-scale tabular data. This underscores the potential of hybrid AI models over single algorithms.

Along with the limitations, our work also points out some that should be discussed. The dataset being drawn from one company might restrict us from generalizing our findings, hinting a future research to incorporate more diverse data to enhance the models we built. However, class imbalance was addressed using resampling techniques which also brought the problem of bias through over/under-sampling so it is necessary for us to see if cost-sensitive learning methods could be used as an alternative. One potential limitation is that we did not perform extensive hyperparameter tuning for all the models evaluated. The fact that the hybrid model delivered high-quality results itself does not mean that with a different set of hyperparameters, another model such as random forest, XGBoost or even another one could excel the hybrid model. Our search for the best model was based on the initial outcomes and we focused the optimization process on the best-performing combination of hybrid variants. Nevertheless, a complete hyperparameter search among all models might probably reveal some other optimized configurations that could

affect the relative performance rankings. Hyperparameter optimization is now a major component of improving the precision of the model. It also emphasizes the importance of increased computing power. One of the improvement areas that needs further consideration is the implementation of an advanced optimization technique like the Bayesian optimization. This might help to gain better control over the model tuning process. The self-training method, even though it produces a decent result with scarcely a labeled data set, still shows the implicit issues of data scarcity in true-life applications. Hence, by expanding the labeled dataset using active learning approaches one could find a useful technique of increasing the accuracy and applicability of the regression model. Model interpretation tools provided valuable actionable explanations for model decisions. While EBM and SHAP aided understandability, model-specific explainers could offer deeper feature importance insights. Our investigation calls for further review of generalizability problems, the emergence of bias mitigation methods and the development of better interpretability.

REFERENCES

[1] J. Hadden, A. Tiwari, R. Roy, and D. Ruta, "Computer assisted customer churn management: State-of-the-art and future trends," *Comput. Oper. Res.*, vol. 34, no. 10, pp. 2902–2917, Oct. 2007.

[2] Y. Xie, X. Li, E. W. T. Ngai, and W. Ying, "Customer churn prediction using improved balanced random forests," *Expert Syst. Appl.*, vol. 36, no. 3, pp. 5445–5449, Apr. 2009.

[3] A. Amin, C. Khan, I. Ali, and S. Anwar, "Customer churn prediction in telecommunication industry: With and without counter-example," in *Proc. Eur. Netw. Intell. Conf.*, Sep. 2014, pp. 134–137.

[4] A. Gallo, "The value of keeping the right customers," *Harvard Bus. Rev.*, Oct. 2014. Accessed: Dec. 10, 2023. [Online]. Available: https://hbr.org/2014/10/the-value-of-keeping-the-right-customers

[5] Y. Ortakci and H. Seker, "Optimising customer retention: An AI-driven personalised pricing approach," *Comput. Ind. Eng.*, vol. 188, Feb. 2024, Art. no. 109920, doi: 10.1016/j.cie.2024.109920.

[6] U. Joy. (2023). *Customer Churn Dataset*. [Online]. Available: https://ieee-dataport.org/documents/customer-churn-dataset

[7] S. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," 2017, *arXiv:1705.07874*.

[8] H. Nori, S. Jenkins, P. Koch, and R. Caruana, "InterpretML: A unified framework for machine learning interpretability," 2019, *arXiv:1909.09223*.

[9] A. K. Ahmad, A. Jafar, and K. Aljoumaa, "Customer churn prediction in telecom using machine learning in big data platform," *J. Big Data*, vol. 6, no. 1, pp. 1–18, Dec. 2019.

[10] A. A. Jamjoom, "The use of knowledge extraction in predicting customer churn in B2B," *J. Big Data*, vol. 8, no. 1, pp. 1–14, Dec. 2021.

[11] Y. Li, B. Hou, Y. Wu, D. Zhao, A. Xie, and P. Zou, "Giant fight: Customer churn prediction in traditional broadcast industry," *J. Bus. Res.*, vol. 131, pp. 630–639, Jul. 2021.

[12] C. Colot, P. Baecke, and I. Linden, "Leveraging fine-grained mobile data for churn detection through essence random forest," *J. Big Data*, vol. 8, no. 1, p. 63, 2021.

[13] H. Kwon, H. H. Kim, J. An, J.-H. Lee, and Y. R. Park, "Lifelog data-based prediction model of digital health care app customer churn: Retrospective observational study," *J. Med. Internet Res.*, vol. 23, no. 1, Jan. 2021, Art. no. e22184.

[14] J. Kozak, K. Kania, P. Juszczuk, and M. Mitrega, "Swarm intelligence goal-oriented approach to data-driven innovation in customer churn management," *Int. J. Inf. Manage.*, vol. 60, Oct. 2021, Art. no. 102357.

[15] H. Sayed and M. A. S. Kholief, "Predicting potential banking customer churn using apache spark ML and MLlib packages: A comparative study," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 11, pp. 1–16, 2018.

[16] N. N. Y. Vo, S. Liu, X. Li, and G. Xu, "Leveraging unstructured call log data for customer churn prediction," *Knowl.-Based Syst.*, vol. 212, Jan. 2021, Art. no. 106586.

[17] A. A. Q. Ahmed, "Churn prediction on huge telecom data using hybrid firefly-particle swarm optimization algorithm based classification," *IOSR-JCE*, vol. 19, no. 4, pp. 30–39, 2017.

[18] A. De Caigny, K. Coussement, K. W. De Bock, and S. Lessmann, "Incorporating textual information in customer churn prediction models based on a convolutional neural network," *Int. J. Forecasting*, vol. 36, no. 4, pp. 1563–1578, Oct. 2020.

[19] A. B. Zoric, "Predicting customer churn in banking industry using neural networks," *Interdiscipl. Description Complex Syst.*, vol. 14, no. 2, pp. 116–124, 2016.

[20] R. O. Q. Dias. (2021). *Predict Churn Using the Udacity's Sparkify Dataset*. [Online]. Available: https://github.com/reneoctavio/sparkify/blob/main/Sparkify-Full.ipynb

[21] A. Li. (2019). *Predicting Churns With Spark*. [Online]. Available: https://github.com/angang-li/sparkify

[22] V. Sharaf. (2021). *Sparkify Churn Prediction With Pyspark on Big Data*. [Online]. Available: https://github.com/zaveta/Churn-Prediction-with-PySpark-Sparkify

[23] A. Famili, W. Shen, R. Weber, and E. Simoudis, "Data preprocessing and intelligent data analysis," *Intell. Data Anal.*, vol. 1, nos. 1–4, pp. 3–23, 1997.

[24] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, "Big data analytics on apache spark," *Int. J. Data Sci. Anal.*, vol. 1, no. 3, pp. 145–164, 2016.

[25] K. Potdar, T. S., and C. D., "A comparative study of categorical variable encoding techniques for neural network classifiers," *Int. J. Comput. Appl.*, vol. 175, no. 4, pp. 7–9, Oct. 2017.

[26] S. García, J. Luengo, and F. Herrera, *Data Preprocessing in Data Mining*, vol. 72. Cham, Switzerland: Springer, 2015.

[27] A. L. Blum and P. Langley, "Selection of relevant features and examples in machine learning," *Artif. Intell.*, vol. 97, nos. 1–2, pp. 245–271, Dec. 1997.

[28] J. Li, K. Cheng, S. Wang, F. Morstatter, R. P. Trevino, J. Tang, and H. Liu, "Feature selection: A data perspective," *ACM Comput. Surv.*, vol. 50, no. 6, pp. 1–45, 2017.

[29] M. L. McHugh, "The chi-square test of independence," *Biochemia Medica*, vol. 23, no. 2, pp. 143–149, 2013.

[30] S. Özşen, "Classification of sleep stages using class-dependent sequential feature selection and artificial neural network," *Neural Comput. Appl.*, vol. 23, no. 5, pp. 1239–1250, Oct. 2013.

[31] T. Rückstieß, C. Osendorfer, and P. van der Smagt, "Sequential feature selection for classification," in *AI 2011: Advances in Artificial Intelligence*. Cham, Switzerland: Springer, 2011, pp. 132–141.

[32] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Mach. Learn.*, vol. 63, no. 1, pp. 3–42, Apr. 2006.

[33] M. Ahsan, M. Mahmud, P. Saha, K. Gupta, and Z. Siddique, "Effect of data scaling methods on machine learning algorithms and model performance," *Technologies*, vol. 9, no. 3, p. 52, Jul. 2021.

[34] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[35] Q. Dong, S. Gong, and X. Zhu, "Imbalanced deep learning by minority class incremental rectification," 2018, *arXiv:1804.10851*.

[36] K. Napierala and J. Stefanowski, "Types of minority class examples and their influence on learning classifiers from imbalanced data," *J. Intell. Inf. Syst.*, vol. 46, no. 3, pp. 563–597, Jun. 2016.

[37] V. H. Barella, L. P. F. Garcia, M. C. P. de Souto, A. C. Lorena, and A. C. P. L. F. de Carvalho, "Assessing the data complexity of imbalanced datasets," *Inf. Sci.*, vol. 553, pp. 83–109, Apr. 2021.

[38] T. Hasanin and T. Khoshgoftaar, "The effects of random undersampling with simulated class imbalance for big data," in *Proc. IEEE Int. Conf. Inf. Reuse Integr. (IRI)*, Jul. 2018, pp. 70–79.

[39] V. M. Calo, Y. Efendiev, J. Galvis, and G. Li, "Randomized oversampling for generalized multiscale finite element methods," *Multiscale Model. Simul.*, vol. 14, no. 1, pp. 482–501, Jan. 2016.

[40] M. S. Santos, P. H. Abreu, P. J. García-Laencina, A. Simão, and A. Carvalho, "A new cluster-based oversampling method for improving survival prediction of hepatocellular carcinoma patients," *J. Biomed. Informat.*, vol. 58, pp. 49–59, Dec. 2015.

[41] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.

[42] W. Zucchini, "An introduction to model selection," *J. Math. Psychol.*, vol. 44, no. 1, pp. 41–61, 2000.
[43] Y. Xu and R. Goodacre, "On splitting training and validation set: A comparative study of cross-validation, bootstrap and systematic sampling for estimating the generalization performance of supervised learning," *J. Anal. Test.*, vol. 2, no. 3, pp. 249–262, Jul. 2018.
[44] H. Sak, A. Senior, and F. Beaufays, "Long short-term memory recurrent neural network architectures for large scale acoustic modeling," in *Proc. Interspeech*, Sep. 2014, pp. 338–342.
[45] R. Dey and F. M. Salem, "Gate-variants of gated recurrent unit (GRU) neural networks," in *Proc. IEEE 60th Int. Midwest Symp. Circuits Syst. (MWSCAS)*, Aug. 2017, pp. 1597–1600.
[46] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–9.
[47] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794.
[48] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 6639–6649.
[49] M. W. Gardner and S. R. Dorling, "Artificial neural networks (the multilayer perceptron)—A review of applications in the atmospheric sciences," *Atmos. Environ.*, vol. 32, nos. 14–15, pp. 2627–2636, Aug. 1998.
[50] C.-Y.-J. Peng, K. L. Lee, and G. M. Ingersoll, "An introduction to logistic regression analysis and reporting," *J. Educ. Res.*, vol. 96, no. 1, pp. 3–14, Sep. 2002.
[51] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
[52] A. Rai, "Explainable AI: From black box to glass box," *J. Acad. Marketing Sci.*, vol. 48, no. 1, pp. 137–141, Jan. 2020.
[53] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities," *Proc. Nat. Acad. Sci. USA*, vol. 79, no. 8, pp. 2554–2558, 1982.
[54] M. Abadi et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. [Online]. Available: https://www.tensorflow.org/
[55] J. Burez and D. Van den Poel, "Handling class imbalance in customer churn prediction," *Expert Syst. Appl.*, vol. 36, no. 3, pp. 4626–4636, Apr. 2009.
[56] J. D. Rodriguez, A. Perez, and J. A. Lozano, "Sensitivity analysis of k-Fold cross validation in prediction error estimation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 3, pp. 569–575, Mar. 2010.
[57] (2019). *Telco Customer Churn (11.1.3+)*. [Online]. Available: https://community.ibm.com/community/user/businessanalytics/blogs/steven-macko/2019/07/11/telco-customer-churn-1113
[58] (2022). *Flight Price Predict. Dataset*. [Online]. Available: https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction
[59] P. Liashchynskyi and P. Liashchynskyi, "Grid search, random search, genetic algorithm: A big comparison for NAS," 2019, *arXiv:1912.06059*.
[60] K. E. Hoque and H. Aljamaan, "Impact of hyperparameter tuning on machine learning models in stock price forecasting," *IEEE Access*, vol. 9, pp. 163815–163830, 2021, doi: 10.1109/ACCESS.2021.3134138.
[61] T. O'Malley, E. Bursztein, J. Long, F. Chollet, H. Jin, and L. Invernizzi, "KerasTuner," 2019. [Online]. Available: https://github.com/keras-team/keras-tuner
[62] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6765–6816, 2017.
[63] Z. H. Hoo, J. Candlish, and D. Teare, "What is an roc curve?" *Emergency Med. J.*, vol. 34, no. 6, pp. 357–359, Jun. 2017.
[64] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in *Proc. 33rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, vol. 33, no. 1, Cambridge, MA, USA, Jun. 1995, pp. 189–196, doi: 10.3115/981658.981684.
[65] H. M. Nguyen, E. W. Cooper, and K. Kamei, "Borderline over-sampling for imbalanced data classification," *Int. J. Knowl. Eng. Soft Data Paradigms*, vol. 3, pp. 4–21, Jul. 2009.
[66] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *SIGKDD Explor.*, vol. 6, pp. 20–29, Jun. 2004.
[67] D. Wilson, "Asymptotic properties of nearest neighbor rules using edited data," *IEEE Trans. Syst. Man Cybern.*, vols. SMC-2, pp. 408–421, 1972.

USMAN GANI JOY received the B.Sc. degree in computer science and engineering from Port City International University (PCIU), in 2020. He is currently pursuing the master's degree in computer science and engineering with East Delta University. He is also a Freelancer specializing in machine learning, data analysis, and blockchain development. His research interests include machine learning, time series forecasting, blockchain, cryptocurrency, and deep learning.

KAZI EKRAMUL HOQUE received the B.Sc. degree in computer science and engineering from Chittagong University of Engineering and Technology (CUET), in 2015, and the master's degree in computer science from the Department of Information and Computer Science (ICS), King Fahd University of Petroleum and Minerals (KFUPM). He is currently an Assistant Professor with East Delta University (EDU). His research interests include machine learning, time series forecasting, software security, empirical software engineering, and deep learning.

MOHAMMED NAZIM UDDIN (Member, IEEE) received the B.S. and M.S. degrees in computer science from the University of Pune, India, in 1998 and 2001, respectively, and the Ph.D. degree in computer science and engineering from Inha University, South Korea, in 2012. He was the Head of the Platform Laboratory, Samsung R&D Institute, Bangladesh, from 2012 to 2015. He has been with the School of Science, Engineering & Technology, East Delta University, as the Associate Dean, since 2015. His research interests include artificial intelligence, data mining, big data analysis, and the Internet of Things.

LINKON CHOWDHURY received the B.Sc. degree in computer science and engineering from the University of Chittagong, Chittagong, Bangladesh, in 2011, and the master's degree (by Research) from Chittagong University of Engineering and Technology, Chittagong, in 2017. He is currently an Assistant Professor with the Department of Computer Science and Engineering, East Delta University, Chittagong. His research interests include bioinformatics, data science, artificial intelligence, and explainable artificial intelligence (XAI).

SEUNG-BO PARK received the B.S. and M.S. degrees in electrical engineering and the Ph.D. degree in computer information engineering from Inha University, South Korea, in 1995 and 1997, respectively. He is currently an Assistant Professor with Inha University. He was with Daewoo Electronics as an Engineering Researcher. His research interests include video story analyzing, semantic contents, video knowledge representation, social network analysis, and AI.